

Full Length Article

Multi-agent contrastive exploration via value decomposition discrepancy

Siying Wang^{a,b,c}, Hongfei Du^d, Chiyu Cai^e, Yang Zhou^d, Wenyu Chen^d, Jinliang Shao^{a,f}, Yuhua Cheng^{a,*}

^a School of Automation Engineering, University of Electronic Science and Technology of China, Chengdu, China

^b College of Computer Science and Artificial Intelligence, Southwest Minzu University, Chengdu, China

^c Intelligent Perception and Control Key Laboratory of Sichuan Province, Sichuan University of Science & Engineering, Zigong, China

^d School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China

^e State Information Center, Beijing, China

^f Tianfu Jiangxi Laboratory, Chengdu, China

ARTICLE INFO

Keywords:

Deep reinforcement learning
Multi-agent cooperation
Exploration and exploitation
Value decomposition

ABSTRACT

Value decomposition, as a factorization approach in multi-agent reinforcement learning (MARL), has been influential in the development of many effective value-based algorithms. Existing studies on value decomposition that focus on representation capability often suffer from low sample efficiency, while other factorization approaches may also have limited representation capability, which may still prevent collaborators from discovering the optimal joint action. To enhance agents' exploration with an unlimited joint state-value function, we propose a Multi-Agent Contrastive Exploration method (MACE) leveraging value decomposition discrepancies and contrastive principles. MACE determines update weights based on the discrepancy between the different value decomposition estimates to set update weights and introduces this difference as an intrinsic target in the update process. Additionally, MACE designs an exploration preference network inspired by this difference, explicitly adjusting the exploration preferences of agents during interactions. Through the experiments of various types and difficulties on Matrix Games and Starcraft Multi-Agent Challenge, we show that MACE not only significantly outperforms the baselines in learning speed and final performance, but also effectively maintains the higher expressivity, representing an innovative solution that integrates the advantages of existing algorithms.

1. Introduction

In order to address the various tasks that involve cooperation and collaboration among multiple agents (Angelotti & Díaz-Rodríguez, 2023; Zhang et al., 2023b), Multi-Agent Reinforcement Learning (MARL) (Hernandez-Leal et al., 2019; Jaques et al., 2019; Mei et al., 2023) which is rooted in reinforcement learning methods (Sutton & Barto, 2018), has seen widespread success in areas such as coordination of robots (Hüttenrauch et al., 2017; Nguyen et al., 2020; Perrusquía et al., 2021), autonomous Unmanned Aerial Vehicles (UAVs) (Chen et al., 2023; Xiao et al., 2023b; Xu et al., 2022; Zhang et al., 2023c), and traffic signal control (Wu et al., 2022; Xiao et al., 2023a; Yang et al., 2023; Zhou et al., 2022). Such tasks demand that agents execute distributed, sequential decision-making processes, ensuring collaboration with one another to achieve a common objective. Nonetheless, developing effective cooperative strategies among agents proves difficult because of partial observability and the non-stationary nature caused by evolving agent policies (Oliehoek & Amato, 2016; Tan, 1993). To overcome these challenges,

the Centralized Training with Decentralized Execution (CTDE) approach has been widely implemented.

As indicated by its designation, CTDE-based MARL algorithms centralize policy training using the global state, but decentralize action execution based on their local observations, which mitigates issues of non-stationarity and training instability MADDPG (Lowe et al., 2017) is a notable algorithm that combines the individual critics into a unified joint critic, with individual policy networks. The core idea is to utilize the observations and actions of all agents as inputs to the value function, allowing the derivation of Q-values. Essentially, it leverages global information from the environment to train policies. By incorporating all agents' observations and actions, each agent's decisions consider the influence of others, fostering cooperation toward shared objectives. The direct inspiration given by MADDPG is to directly integrate all the agent information together during the training phase. While its essence is to search for the optimal joint actions of agents in the parameter space composed of all the observations and actions (Hu et al., 2023; Rashid et al., 2020b). This approach has given rise to a type of CTDE method,

* Corresponding author.

E-mail addresses: siyingwang@uestc.edu.cn (S. Wang), duhongfei@ydnkj.com (H. Du), caicy@sic.gov.cn (C. Cai), zhouy@std.uestc.edu.cn (Y. Zhou), cwy@uestc.edu.cn (W. Chen), jinliangshao@uestc.edu.cn (J. Shao), yhcheng@uestc.edu.cn (Y. Cheng).

<https://doi.org/10.1016/j.neunet.2026.108929>

Received 8 December 2024; Received in revised form 3 March 2026; Accepted 31 March 2026

Available online 6 April 2026

0893-6080/© 2026 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

represented by QTRAN (Son et al., 2020, 2019), which assumes that tasks can be decomposed into sub-tasks for each agent, and the set of optimal actions for each agent corresponds to the overall optimal joint actions. Under certain conditions, this method can optimize both the optimal joint state-action value function Q_{jt} and the utility functions Q_i of individual agents, ensuring effective updates to the cooperative policy. However, as the number of agents increases, optimizing the joint policy becomes increasingly difficult due to the exponential expansion of the joint action space (Hu et al., 2023; Lowe et al., 2017). Directly finding the optimal joint action in such a parameter search space is undeniably a formidable challenge (Pu et al., 2021; Rashid et al., 2020a; Shen et al., 2022).

Another form of CTDE policy involves designing specialized network architectures to minimize the parameter search space, accelerating the training of cooperative policies. These techniques are categorized under Value Decomposition (VD) or Value Factorization, where the surrogate joint state-action value function Q_{tot} is factorized into individual utility functions Q_i for each agent. A key to effective updates in these methods is ensuring that the optimal joint action corresponds to the aggregation of each agent's greedy action, which adheres to the Individual-Global-Max (IGM) principle, and then it can monotonically synchronously optimize the individual value functions of each agent when optimizing the centralized value function. VDN (Sunehag et al., 2017) and QMIX (Rashid et al., 2018) are prominent VD methods that conform to the IGM principle. Due to their excellent sample efficiency and ease of implementation, they are considered foundational for later value factorization techniques. Nonetheless, their decomposition mechanism only meets the sufficient condition of IGM (Son et al., 2019), limiting the expressive power of the joint action-value function. Due to the expressive limitations of neural networks, there is only a positive correlation between the joint state-action function and the utility function, making it difficult for this type of method to learn the correct factorization, especially in the relative over-generalization problem (Böhmer et al., 2020; Wei et al., 2018), they still suffer from sample-inefficiency.

This sample efficiency plays a vital role in evaluating model training within the CTDE framework of MARL, where higher efficiency generally implies that the model demonstrates superior performance during training. One intuitive idea is to combine the strengths of the two aforementioned VD methods. The estimates of these two decomposition approaches may be different, and this difference reflects the variance in their corresponding cooperative policies. Since Q_{jt} is capable of higher expressivity, its correct estimation can guide the proper update of Q_{tot} , and this difference implicitly indicates the direction in which the joint collaborative policy should explore. Proper exploration allows the model to swiftly identify the optimal joint actions in a vast action space, correcting the policy network's misestimated values and speeding up the learning process of the whole cooperative policy.

In this paper, we propose a Multi-Agent Contrastive Exploration approach that leverages VD discrepancy for exploration and policy update, which is grounded in the concept of contrastive learning. Specifically, the MACE architecture contains two value function estimators, each responsible for estimating the joint state-action value functions corresponding to the two VD approaches, Q_{jt} and Q_{tot} . As Q_{jt} possesses the higher expressivity to accurately approximate the reward distribution and the optimal joint actions, MACE leverages the difference between Q_{jt} and Q_{tot} to create an implicit reward function and a weighting mechanism that guides exploration, which is used to update the two internal function estimators. Additionally, MACE introduces an action preference network driven by the average policy entropy of each agent, incentivizing agents to make a broader range of action choices. This approach ensures that actions with higher Q-values are prioritized for sampling, while actions with smaller Q-values still have the opportunity to be sampled, enhancing the exploratory behavior. This mechanism is quite similar to the process of policy co-guiding (Luo et al., 2023). It occurs during the estimation of different joint state-action value functions but is orthogonal to methods like pretraining or fine-tuning (Lee et al.,

2022; Xie et al., 2021; Zhang et al., 2023a). We design experiments of varying difficulty levels in the Matrix Games and Starcraft Multi-Agent Challenge, to validate the superiority of the proposed MACE method. The results demonstrate that MACE not only accurately estimates the reward distribution but also exhibits superior performance. In general, our main contributions are summarized in following three folds:

- We introduce a novel MARL joint policy training approach that leverages the estimation discrepancies across different types of VD methods to boost exploration and guide the joint policy update.
- MACE incorporates an action preference network grounded in the average policy entropy of all agents, enabling more effective action sampling and thereby strengthening exploration.
- We verify the effectiveness of the proposed MACE method through rigorous experiments on a wide range of levels and tasks, in comparison with several baseline methods.

The remaining sections of this paper are structured as follows: Section II reviews related literature, and Section III outlines the MARL preliminaries. In Section IV, we introduce the MACE algorithm and provide a detailed comparison of the advantages and limitations of two types of VD techniques. Section V reports the experimental outcomes and ablation studies, with conclusions drawn in Section VI.

2. Related works

In methods for training agent cooperation, VD is an efficient training paradigm that can effectively address problems such as partial observability, algorithmic instability, and credit assignment in cooperative environments. These methods primarily focus on designing effective training and task decomposition approaches that satisfy the Individual-Global-Maximum (IGM) (Son et al., 2019) principles. All these methods are divided into two categories based on how they satisfy the IGM condition: one relies on structural constraints of the network, while the other focuses on augmenting information and reducing weights.

The first class of methods imposes structural constraints like *additivity* or *monotonicity* on the network to decompose the joint action-value function. Consistent with the IGM principle, VDN (Sunehag et al., 2017) decomposes Q_{tot} into the summation of individual value functions $\sum_i Q_i$, without leveraging extra information available during training. While QMIX (Rashid et al., 2018) enforces monotonicity constraints on the mixing network, ensuring that both Q_{tot} and Q_i adhere to the IGM principle. Qatten (Yang et al., 2020) develops a theoretical method for decomposing joint Q-values into local Q-values, regardless of the number of agents, without requiring additional conditions. It employs multi-head attention to combine individual agent utilities with global data. DVD (Li et al., 2022) introduces a unique causal reasoning approach to VD, isolating the confusion path from the global state by using trajectory graphs based solely on local trajectories as the confusion factor. GraphMIX (Naderalizadeh et al., 2020) incorporates GNN and attention mechanisms into QMIX, with agents represented as nodes in a directed graph, and attention controls the edge weights to capture agent relationships. VGN (Wei et al., 2022) factors in the influence of agents with minimal Q-values and introduces graph attention networks to establish dynamic agent interactions. Each agent's decomposition factor is shaped by the reward signals from all other relevant agents. TransMix (Khan et al., 2022), a Transformer-based joint action-value network, focuses on capturing global and local interactions among Q_i , past trajectories, and global states, enabling effective attention to key states and handling long time horizons. DVF (Huang et al., 2024) transforms the IGM principle to difference value factorization, which addresses non-monotonic constraints by ensuring the consistency of the IGM process between the joint difference value and the complex non-linear sum of independent difference values. QDAP (Zhao et al., 2024a) introduces a downsizing adaptive policy framework to QMIX to discern relevant values from dead agents, thereby aiding remaining active agents in learning more appropriate cooperative policies. The recent MDQ (Ding et al., 2023a) com-

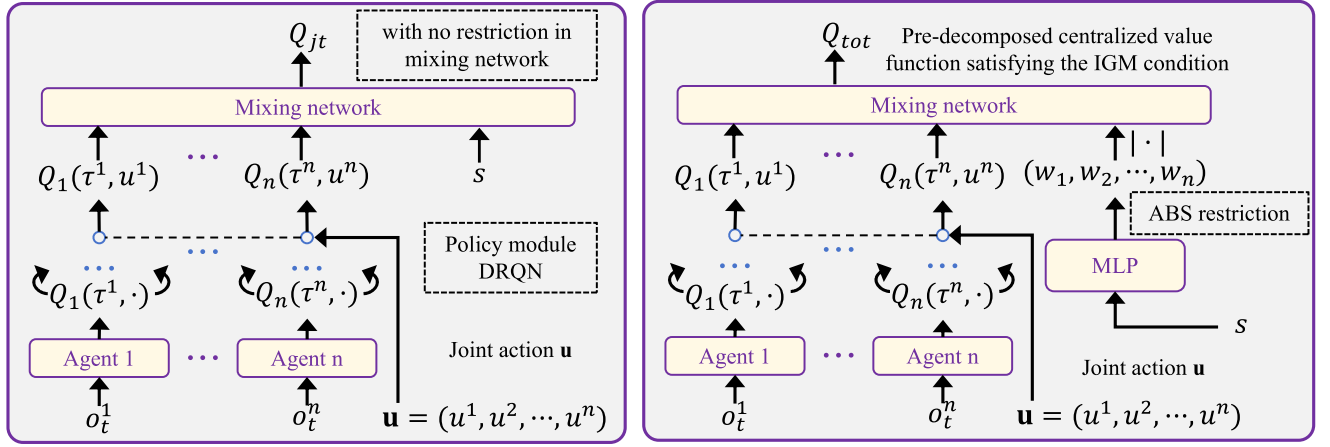


Fig. 1. The illustration of two distinct types of value decomposition methods, (a) on the left with the higher expressivity of the centralized state-action value function, (b) on the right with the only positive representation of the centralized state-action value function.

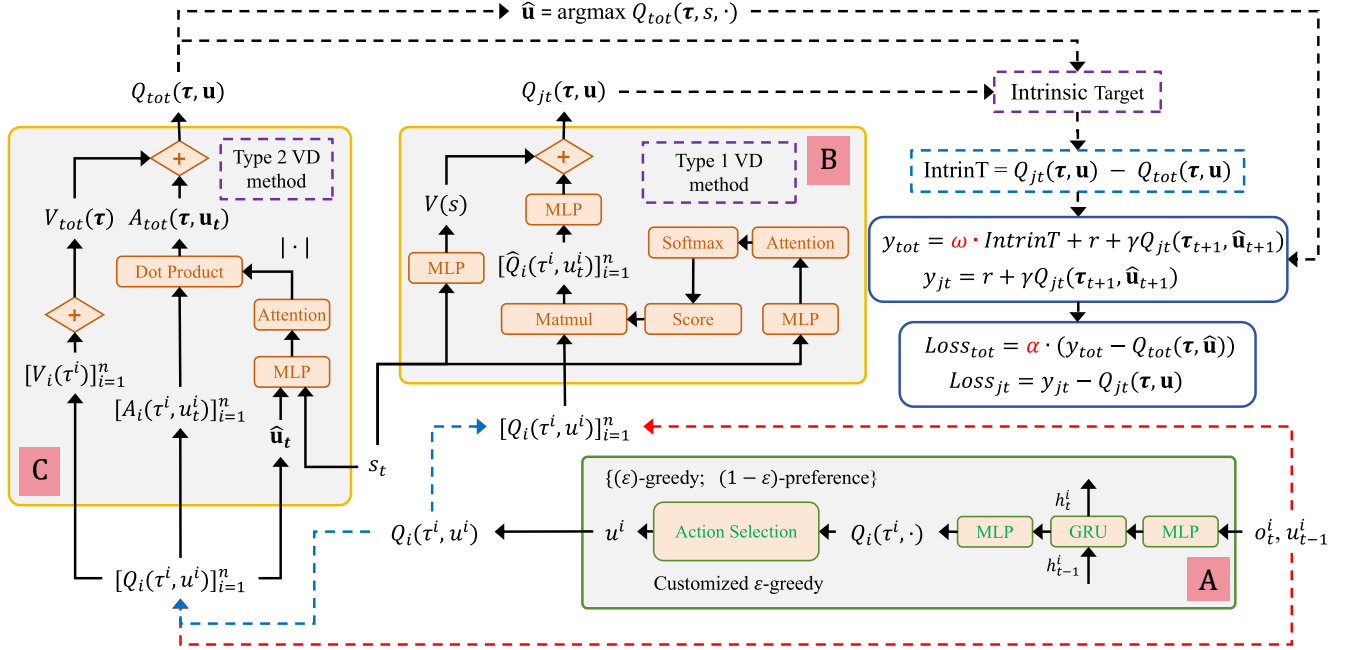


Fig. 2. The illustration of MACE model, (a) agent policy network with action preference, (b) centralized value function part with unconstrained by monotonicity; (c) centralized value function part with restricted representation capability.

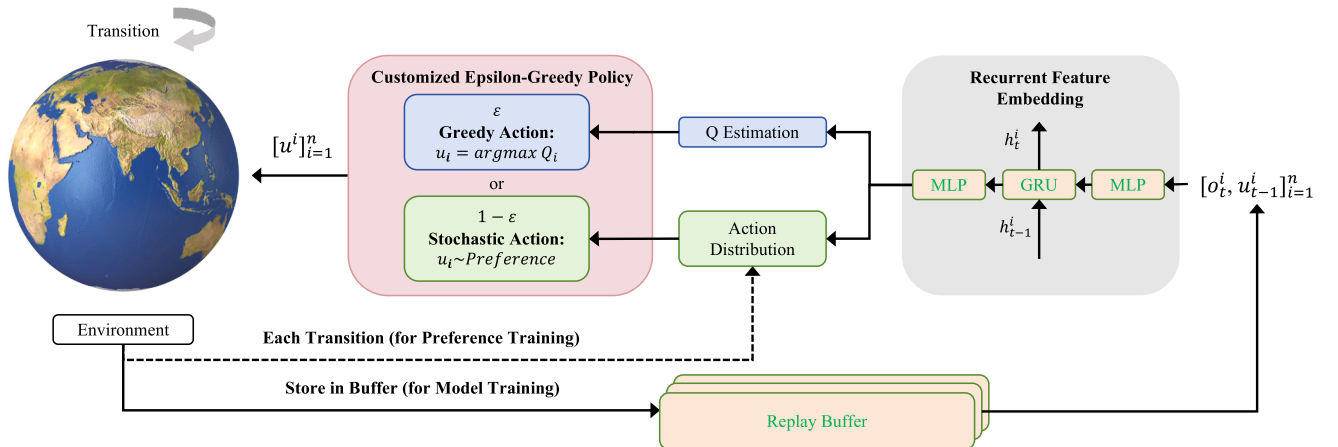


Fig. 3. The illustration of the interaction between agents of MACE and environment.



Fig. 4. The cooperative scenario 2c_vs_64zg and 3s5z_vs_3s6z in SMAC (Wang et al., 2024).

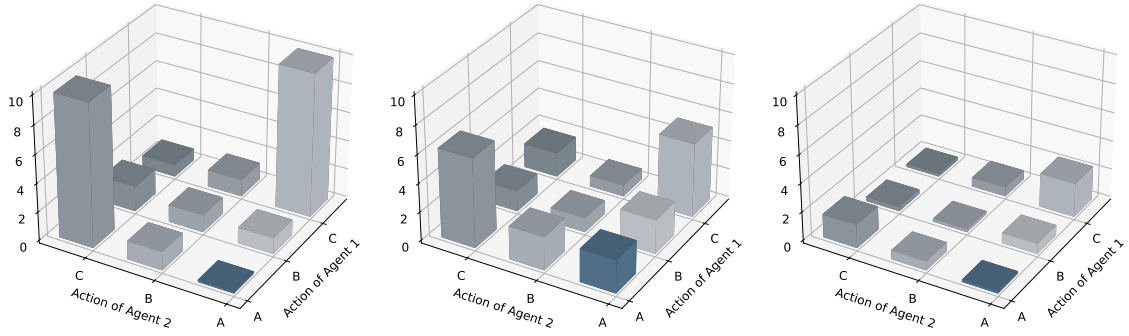


Fig. 5. The histogram of discrepancy value $|Q_{jt} - Q_{tot}|$ (at 2k, 3k, 4k training steps) in near-degenerate Matrix Game.

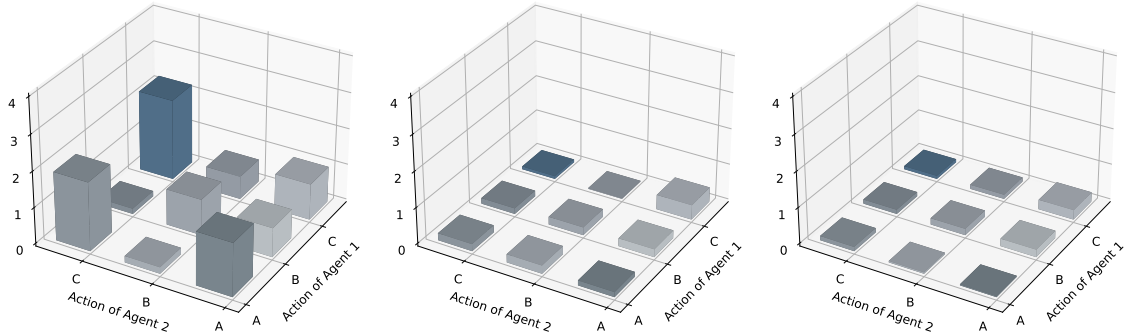


Fig. 6. The histogram of discrepancy value $|Q_{jt} - Q_{tot}|$ (at 2k, 3k, 4k training steps) in stochastic Matrix Game.

combines mean-field theory with VD to address cooperative, competitive, or hybrid tasks. It decomposes the individual value function into local and mean-field components, enabling the integration of personalized rewards in hybrid task settings. Drawing inspiration from the human nervous system, HAVEN (Xu et al., 2023) introduced a two-tier QMIX approach for both inter-level and intra-level coordination. By structuring rewards hierarchically, the advantage function from the higher level feeds into the lower-level reward, ensuring coordination between inter-level and inter-agent policies. Constraining the network structure for VD may reduce the network's expressive capacity, hindering its ability to represent complex value functions.

Another class of approaches approximates satisfying the IGM condition by directly assuming the joint optimal action and reducing the weights of the corresponding non-optimal actions. One representative example is QTRAN (Son et al., 2019), which introduces a soft regularization constraint in the updating process. Building upon QTRAN, QTRAN++ (Son et al., 2020) explicitly specifies the relationship $Q_{tot}(s, \tau, \bar{u}) = Q_{tran}(s, \tau, \bar{u}) > Q_{tran}(s, \tau, u) > Q_{tot}(s, \tau, u)$ and in-

troduces the corresponding loss for Q_{tot} to expedite training, accelerate the learning process, and stabilize the model's performance. While WQMIX (Rashid et al., 2020a) introduces weighted projection to optimize QMIX, reducing the influence of non-optimal joint actions during learning. QPLEX (Wang et al., 2020) transforms the IGM principle into constraints on the advantage function and leverages a dual-competitive structure to decompose Q_{tot} . ResQ (Shen et al., 2022) decomposes the joint Q-value into a primary and a residual function using residual decomposition, mitigating the issue of limited representation in VD. With a similar core, RQN (Shen et al., 2022) develops an individualized factor for each agent, reflecting the importance of specific Q-value trajectories, guiding agents to optimal behavior. While LOMAQ (Zohar et al., 2022) offers a scalable method for VD, aimed at refining credit assignment through the use of local agent rewards. Liu et al. (2023a) introduced a novel Q-value decomposition technique that accounts for other agents' actions, expanding an agent's observations to incorporate both local and observable actions, promoting optimal cooperative behavior. CIA (Liu et al., 2023b) proposes a contrastive identity-aware learning

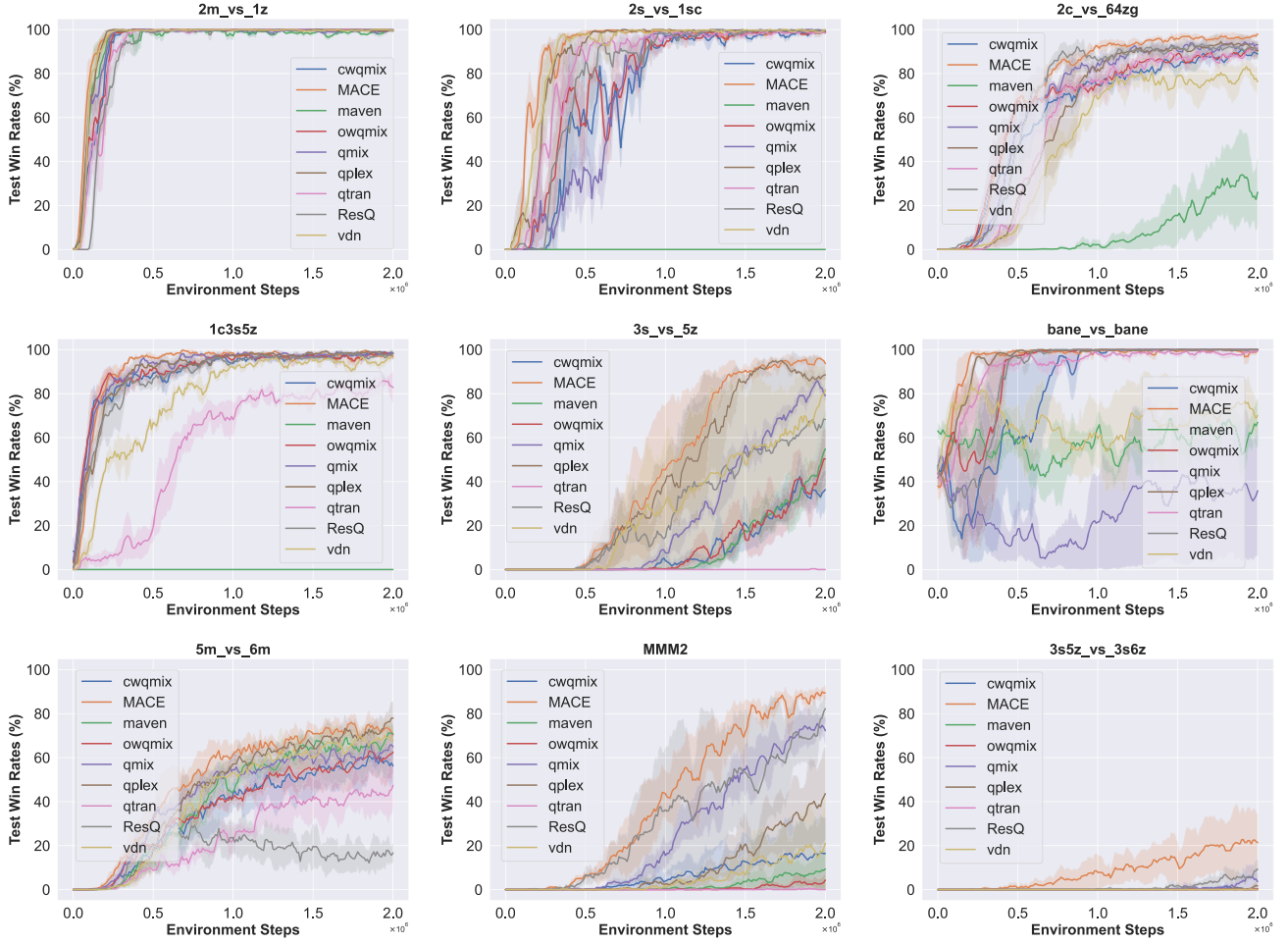


Fig. 7. Training result of MACE and other baseline algorithms in different scenarios of SMAC.

approach to distinctly differentiate between the credit levels assigned to agents. DGN (Jiang et al., 2018) represents the multi-agent environment as a graph, treating each agent as a node with encoded local observations as node features. MARGIN (Ding et al., 2023b) applies graph convolutional networks to gather neighboring agents' information, facilitating high-level feature extraction for enhanced collaboration, which breaks down global mutual information into a weighted sum of local mutual information between input features and hidden feature representations. RA3 (Wan, 2023) incorporates the Anderson Acceleration into QMIX to estimate a more accurate joint state-action function. Similarly, VAOS (Li et al., 2024) proposes an operator-switching method to integrate the advantages of value averaging and to enhance learning stability.

Although these techniques gather richer information from the environment and agents, their effectiveness is hindered by their failure to maintain IGM consistency and the need for extra counterfactual information. Therefore, our proposed MACE amalgamates the strengths of the aforementioned VD methods, aiming to achieve a highly sample-efficient and effective exploration approach.

3. Preliminaries

3.1. Dec-POMDPs

We consider the cooperation between multiple agents as the system consists of n agents, expressed as $n \in N \equiv 1, \dots, n$, which operate within the constrained observation areas and make decisions by uti-

lizing their individual observation features. Such decision-making processes are typically embodied in the concept of *Decentralized Partially Observable Markov Decision Processes* (Dec-POMDPs), represented and modeled by the tuple $G = (S, U, P, r, Z, O, n, \gamma)$. At each timestep t , the environment transitions according to the function $P(s' | s, \mathbf{u}) : S \times U \times S \rightarrow [0, 1]$, with the actual state $s \in S$ conveys the collective knowledge of the agents and supplementary environmental aspects, and $\mathbf{u} \in U \equiv U^n$ the collective individual action $u^i \in U^i$. All agents share a common reward function $r(s, \mathbf{u}) : S \times U \rightarrow \mathbf{R}$, and each agent obtains a partial observation $z^i \in Z$ via the observation function $O(z^i | s, u^i) : S \times U \rightarrow Z$. The objective of all agents is to collaboratively maximize the cumulative team reward $R_t = \sum_{k=0}^T \gamma^k r_{t+k}$, as defined by the joint value function $Q^\pi(s_t, \mathbf{u}_t) = \mathbb{E}_{s_{t+1:\infty}, \mathbf{u}_{t+1:\infty}} [R_t | s_t, \mathbf{u}_t]$.

3.2. Task and value decomposition

To achieve better coordination, it is generally necessary to clearly define the specific tasks of each cooperating agent, which allows for updating the parameters of the policy network based on the assigned task information. This process, which is called task decomposition, ensures that each agent can effectively coordinate its actions and optimize the overall system performance. In the paradigm of RL, the completion of tasks usually relies on the joint actions of agents, which is determined by the joint action value function Q_{tot} . As a result, the task decomposition inherently corresponds to decomposing the joint action value function. The essence of VD lies in forming a direct link between the joint state-action value function Q_{tot} and the utility functions of individual agents

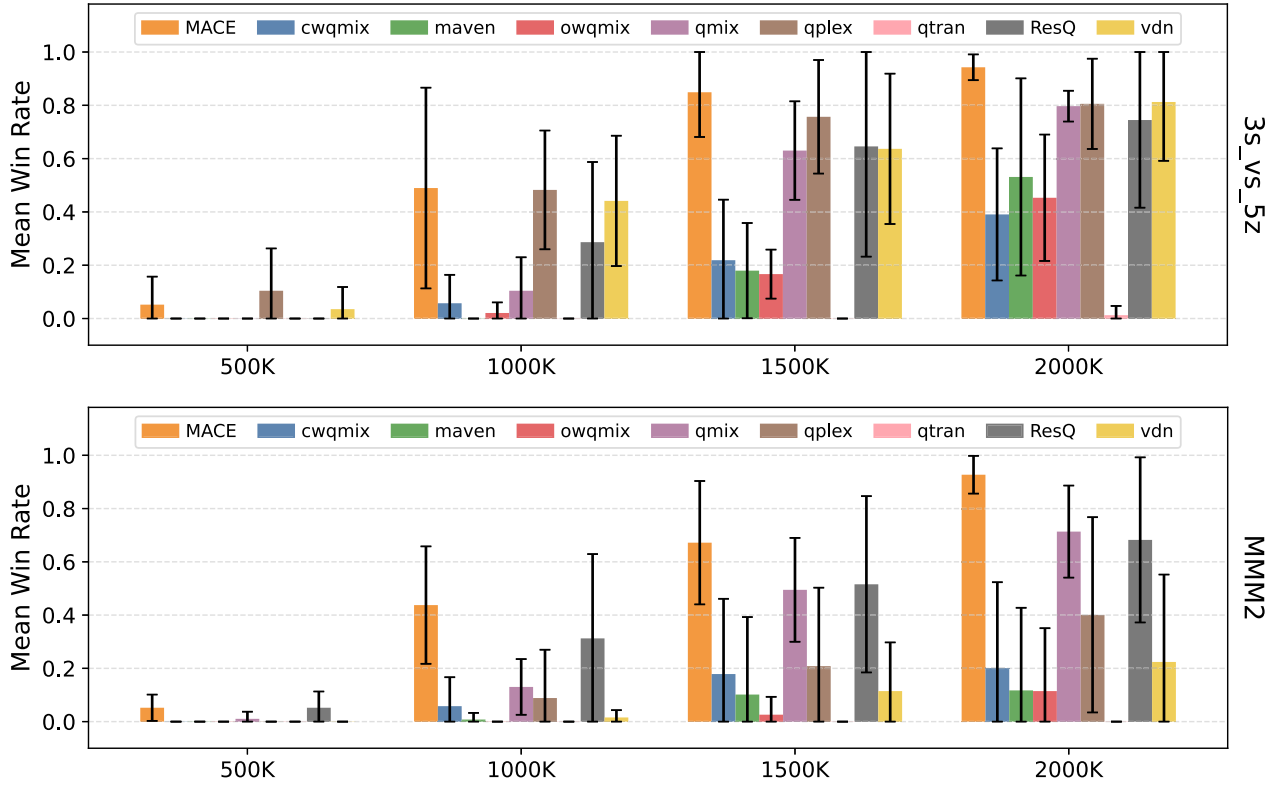


Fig. 8. The multi-seed error bars of MACE and other baselines on 3s_vs_5z and MMM2.

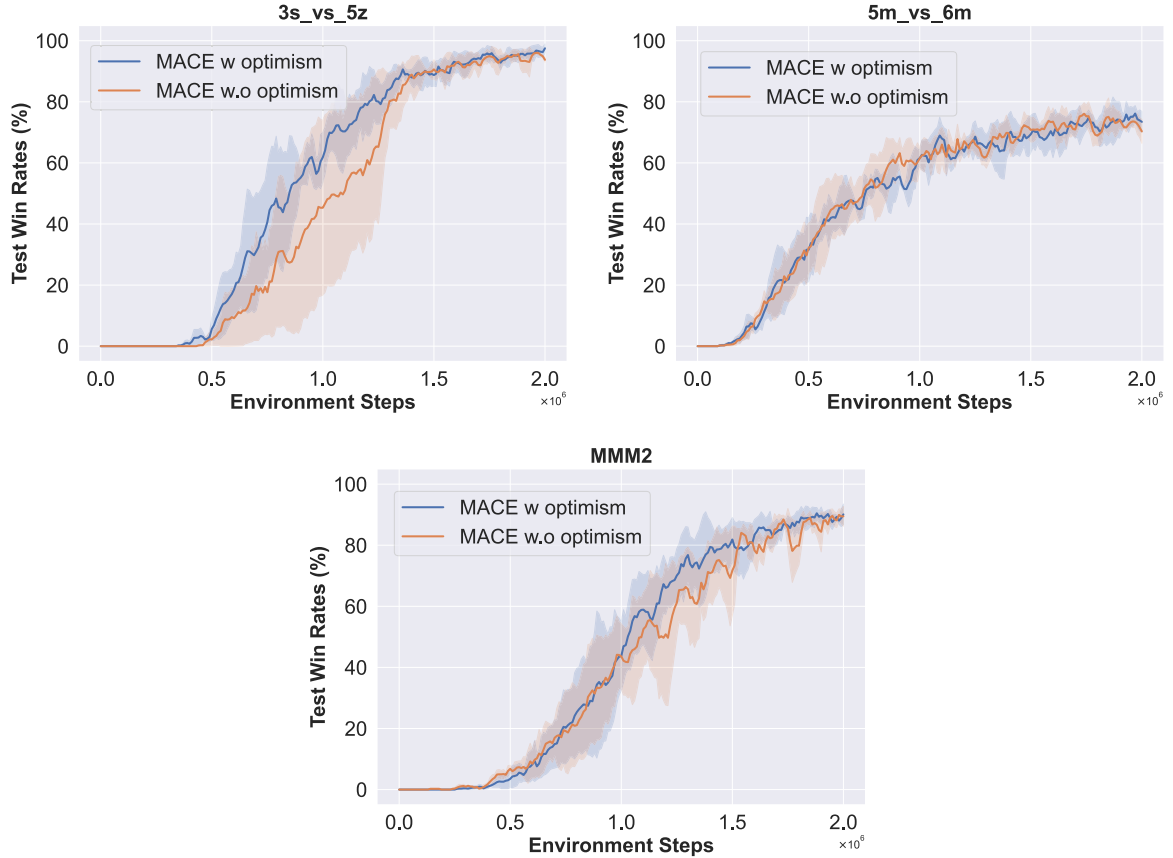


Fig. 9. The effect of additional optimism part in training process of MACE on different SMAC scenarios.

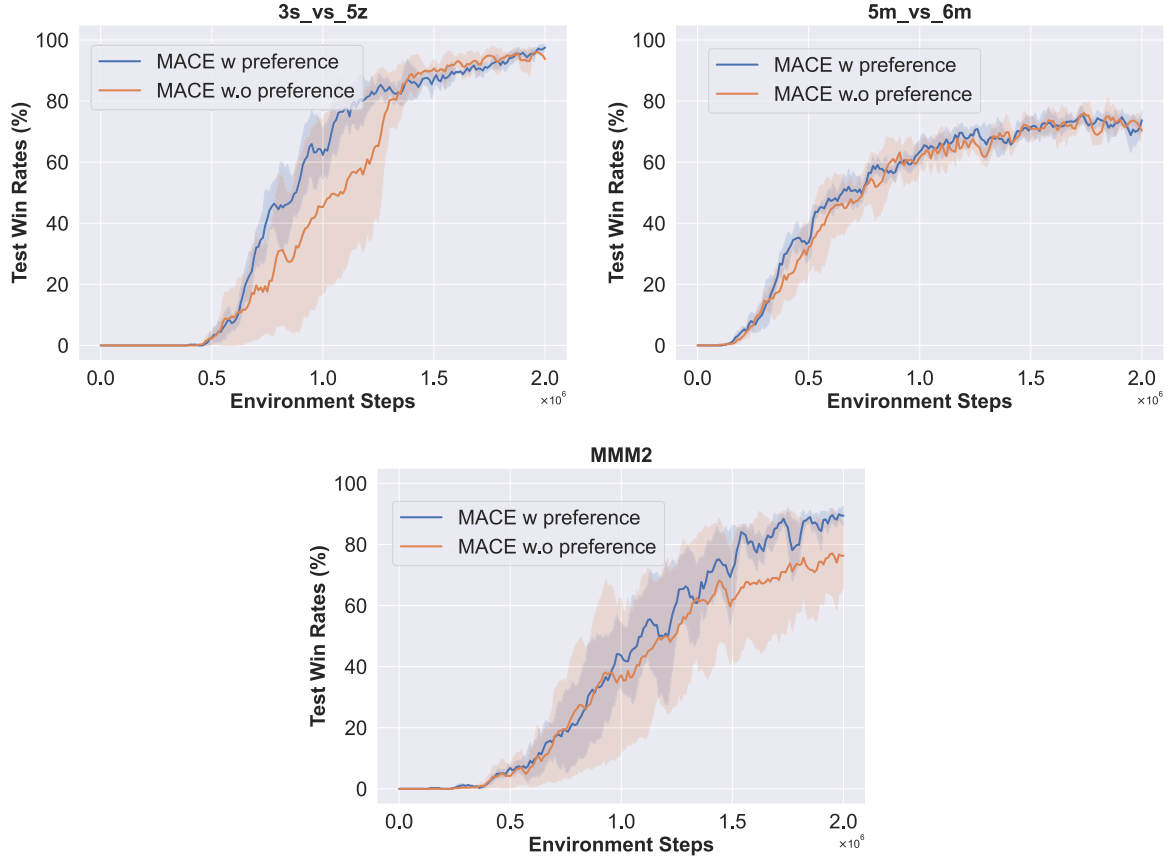


Fig. 10. The effect of action preference network in MACE along with training process.

$[Q_i]_{i=1}^n, i \in N$ during centralized training and satisfying:

$$\arg \max_{\mathbf{u}} Q_{tot}(\boldsymbol{\tau}, \mathbf{u}) = (\arg \max_{u^1} Q_1(\tau^1, u^1) \dots \arg \max_{u^n} Q_n(\tau^n, u^n)). \quad (1)$$

The equation is satisfied when $[Q_i]_{i=1}^n$ adheres to the IGM criterion for the joint-value function Q_{tot} , and the relevant cooperative tasks are termed decomposition-available. Consequently, $Q_{tot}(\boldsymbol{\tau}, \mathbf{u})$ is decomposable into $[Q_i(\tau^i, u^i)]_{i=1}^n$, or $[Q_i]_{i=1}^n$ constitutes a decomposition of Q_{tot} . Two common methods for value function decomposition are VDN (Sunehag et al., 2017) and QMIX (Rashid et al., 2018), which meet the sufficient conditions as follows:

$$\begin{aligned} \text{(Additivity)} \quad Q_{tot}(\boldsymbol{\tau}, \mathbf{u}) &= \sum_{i=1}^n Q_i(\tau^i, u^i) \\ \text{(Monotonicity)} \quad \frac{\partial Q_{tot}(\boldsymbol{\tau}, \mathbf{u})}{\partial Q_i(\tau^i, u^i)} &\geq 0, \quad \forall i \in n \end{aligned} \quad (2)$$

This type of method initially hypothesizes a centralized value function that can be decomposed and proceeds to break it down into a combinatorial structure that incorporates each agent's utility function. QMIX specifically represents Q_{tot} as a monotonic nonlinear combination of individual utilities Q_i through a mixing network, where the parameter weights are all positive, which can be depicted as $Q_{tot}^{QMIX}(\boldsymbol{\tau}, \mathbf{u}) = \sum_{i=1}^n |w_i| Q_i(\tau^i, u^i)$. While VDN can be denoted as a special case of QMIX with a decomposition $Q_{tot}(\boldsymbol{\tau}, \mathbf{u}) = \sum_{i=1}^n Q_i(\tau^i, u^i)$ where $w_i \equiv 1$ and it also satisfies the IGM principle. Typically, these positive parameters can be established by imposing structural constraints on the neural network, which maintains a solely positive correlation between the input utility functions Q_i and the output Q_{tot} . This optimization effectively reduces the time complexity from $\mathcal{O}(|U|^n)$ to $\mathcal{O}(n|U|)$, which facilitates faster learning of effective joint actions by the agents.

The restricted positive correlation between the input and output in these VD methods can lead to Q_{tot} inadequately capturing the reward distribution associated with the task, thereby impeding the agents' capacity to discover optimal joint actions. A different category of VD methods reestimates a true joint action value function, using QMIX for action selection and employing QMIX for action selection and decreasing the update weights of samples with non-optimal actions during training, which encourages cooperative agents to focus more on samples with optimal joint actions in their updates. WQMIX (Rashid et al., 2020a) exemplifies this type of VD method, establishing an updated weight of 1 for samples associated with optimal actions and designating a weight of $\alpha(s, \mathbf{u}) < 1$ for samples linked to non-optimal actions, represented as follows:

$$\alpha(s, \mathbf{u}) = \begin{cases} 1 & y > Q_{jt}(s, \boldsymbol{\tau}, \hat{\mathbf{u}}^*) \text{ or } \mathbf{u} = \hat{\mathbf{u}}^*; \\ \alpha \in (0, 1] & \text{otherwise} \end{cases} \quad (3)$$

where $y := r + \gamma Q_{jt}(s', \boldsymbol{\tau}', \arg \max_{\mathbf{u}'} Q_{tot}(\boldsymbol{\tau}', \mathbf{u}', s'))$ is the estimation of target network. The joint value function Q_{jt} , as fitted by WQMIX without restrictions imposed by the neural network structure, demonstrates superior expressive capability compared to QMIX and VDN. Nevertheless, because these approaches rely on neural networks to estimate the centralized value functions Q_{jt} and Q_{tot} for choosing joint actions, they encounter problems related to sample inefficiency and prolonged training convergence times.

3.3. Policy gradient method

The core concept of policy gradient methods involves creating a policy π governed by neural network parameters ϕ , with updates to these parameters driven by the feedback received from the environment. This

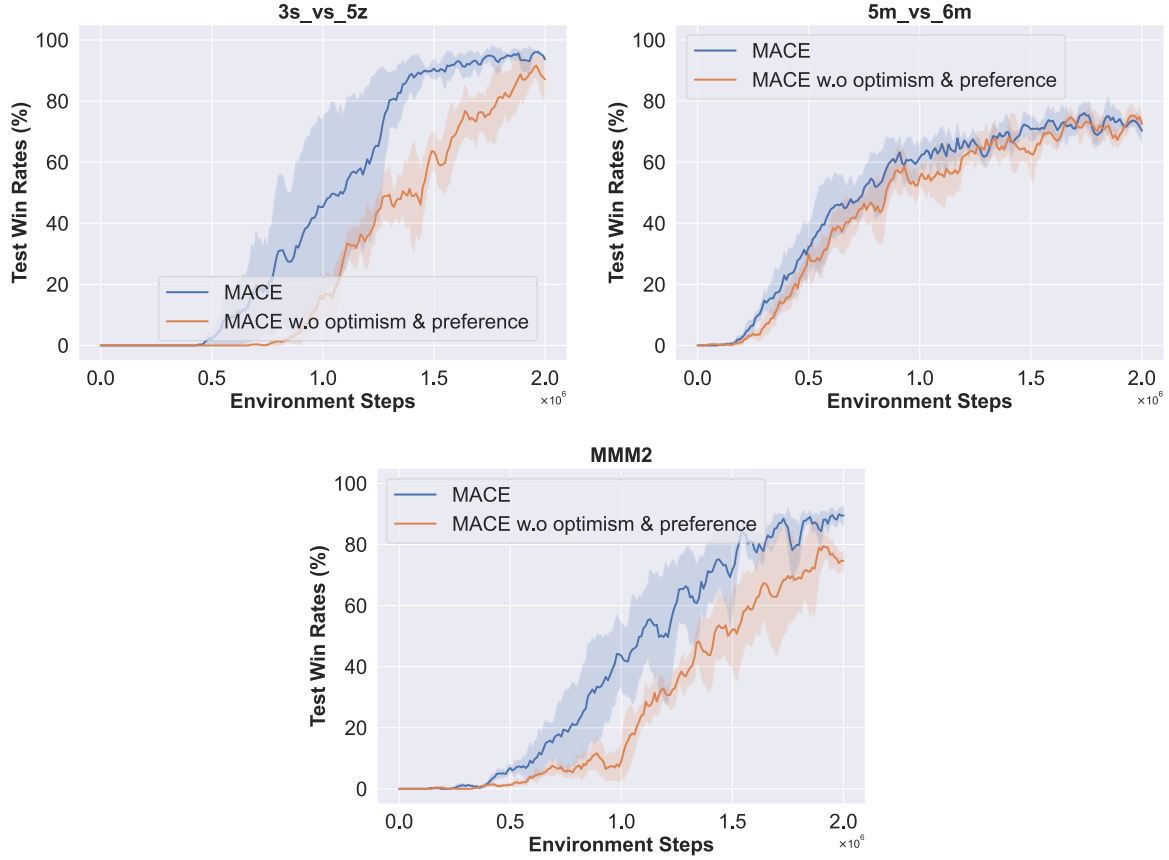


Fig. 11. The effect of no-extra-exploration modules in MACE along with training process.

approach is based on the policy gradient theorem (Sutton et al., 1999), with the gradient update equation formulated as:

$$\nabla_{\phi} \mathcal{L}(\phi) = \mathbb{E}_{s_t \sim \mathcal{P}, a_t \sim \pi} [Q^{\pi}(s_t, a_t) \nabla_{\phi} \log \pi_{\phi}(a_t | s_t)] \quad (4)$$

where $Q^{\pi}(s_t, a_t)$ indicates the estimated Q function with policy π , which is used to fit the accumulative rewards of interactions with the environment. The accuracy of Q estimation determines both the direction and accuracy of policy updates, with a typical technique being to subtract a state value V to assess the relative importance of actions as: $A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$, where A^{π} is called the advantage function. To enhance the exploration and avoid falling into local optima, a common practice is to include the entropy of policy in the objective function, thus modifying the gradient update form:

$$\nabla_{\phi} \mathcal{L}(\phi) = \mathbb{E}_{s_t \sim \mathcal{P}, a_t \sim \pi} [A^{\pi}(s_t, a_t) \nabla_{\phi} \log \pi_{\phi}(a_t | s_t)] + \beta \mathbb{E}_{s_t \sim \mathcal{P}} [\nabla_{\phi} H^{\pi}(s_t)] \quad (5)$$

where $H^{\pi}(s_t) = -\sum_a \pi_{\phi}(a_t | s_t) \log \pi_{\phi}(a_t | s_t)$ represents the Shannon entropy of policy π and β is a temperature parameter that controls the level of stochasticity of π , which can be set to be a constant or optimized with another objective function (Haarnoja et al., 2018a).

4. Multi-agent contrastive exploration

In this section, we will introduce our proposed MACE method in detail. In particular, this section starts by outlining the pros and cons of the two categories of VD techniques, details the components of MACE, and ends with a thorough explanation of how MACE is implemented.

Notation: For simplicity and uniformity of presentation, we use the following symbols to represent their unique connotations. We summarize these symbols in Table 1. Furthermore, we use Q_{ji} to briefly represent $Q_{ji}(\mathbf{r}, \mathbf{u})$ for the sake of convenience, and other symbols without independent variables have the same meaning as this.

4.1. Pros and cons of two distinct types of value decomposition methods: a qualitative description

4.1.1. Value decomposition with unrestricted capability of centralized value functions

As a core component of deep learning, neural networks are often used as feature extractors to express high-level features and abstract information in the observed data. The proficiency of neural networks in accomplishing the aforementioned tasks is attributed to their adoption of a multi-tiered network architecture and diverse activation functions for non-linear transformations of input features. This design allows the network to effectively address non-linear relationships within the data, endowing it with the capability to approximate arbitrary functions (Elfwing et al., 2018; Liang & Srikant, 2017).

Generally, under the CTDE framework, MARL algorithms integrate the utility functions of all agents to consider joint decisions in cooperative scenarios. As depicted in the left part of Fig. 1, the utility function Q_i of the agent, fitted through the DRQN module, enters a mixing network to yield the centralized value function Q_{ji} . The mixing network of this type of VD method typically takes the form of a regular multi-layer feedforward network, and it does not impose any constraints on the weight parameters. Each utility function signifies the selection of an action from the action space of an individual agent, hence the joint action space of Q_{ji} is $(|U|^n)$. The general idea behind such algorithms is to

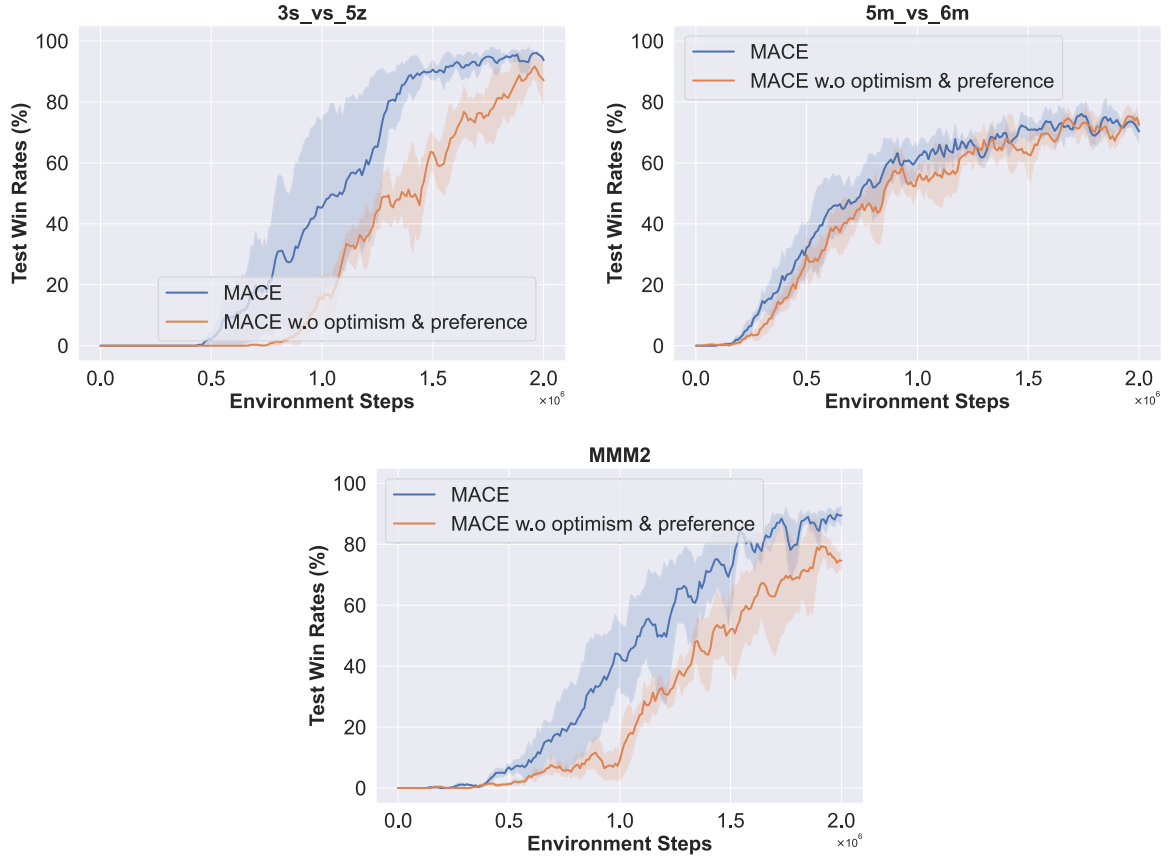


Fig. 12. The effect of different update frequencies of action preference network in MACE along with training process.

Table 1
Notations in MACE.

Notation	Description	Notation	Description
s	The current state (at time t)	S	The set of all states
s'	The next state (at time $t + 1$)	U	The set of all actions
u^i	The action of agent i	N	The set of all agents
$\mathbf{u}$	Joint actions	τ^i	Action-observation history of agent i
o^i	Observation of agent i	$\mathbf{r}$	Joint action-observation histories
$\mathbf{o}$	Joint observation	$r(s, \mathbf{u})$	Team reward
$Q_i(\tau^i, u^i)$	Utility function of agent i	γ	Discount factor
ω	Scaling factor of value decomposition discrepancy	ϵ	Action selection probability of ϵ -greedy
α	The penalty coefficient of the non-optimal joint action	β	The temperature coefficient of the Shannon entropy
η	Action preference network	ϕ	Parameters of η
A_i^η	Advantage function on η of agent i	H^η	Shannon entropy of η
$\mathcal{P}$	Transition function	ξ_{entropy}	Minimum expected entropy
$\mathcal{K}$	Update interval	λ	Learning rate
ρ_i	The weighting coefficient of advantage function for agent i	σ	An arbitrarily small positive number

assume that the centralized value function can be decomposed, which maintains the higher expressivity in the whole process. Nonetheless, the absence of restrictions on the input actions for estimating the centralized value function and utility function prevents a clear determination of the relationship between joint and individual actions, leading to a sluggish optimization of the agents' collaborative decision-making process.

4.1.2. Value decomposition with explicitly-restricted capability of centralized value functions

The expressive capability of a neural network is intricately linked to its weight parameters. Within the CTDE framework, VD methods with restricted expressive capacity face challenges in fitting arbitrary functions. The fundamental issue arises from the absolute value normaliza-

tion of weight parameters generated by the hypernetwork, aimed at ensuring consistency between joint actions and individual actions. Enforcing all weights to be positive leads to a loss of the network's ability to express negative correlations, rendering it incapable of adapting to certain intricately complex function relationships. As illustrated in the right part of Fig. 1, the mixing network integrates the utility functions Q_i using weight parameters after absolute value normalization. This enforces a consistently positive correlation (or a consistently monotonic relationship) between the centralized value function and utility functions, thereby guaranteeing the satisfaction of the IGM condition $\frac{\partial Q_{\text{tot}}(\mathbf{r}, \mathbf{u})}{\partial Q_i(\tau^i, u^i)} \geq 0$.

When all the weights are constrained to be positive, the neural network loses its fitting capability for certain functions due to its inability to represent negative correlations between input utility functions. This de-

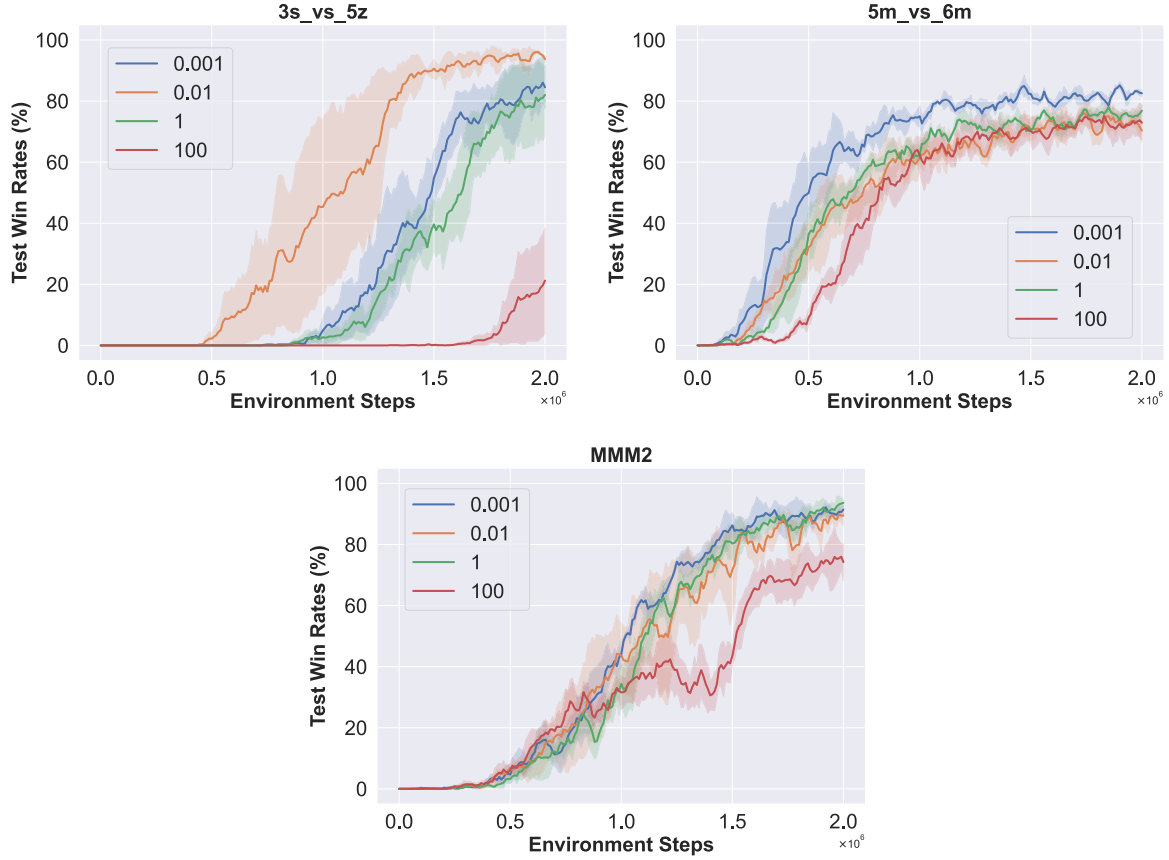


Fig. 13. The effect of parameter ω in MACE along with training process.

efficiency is reflected in the restricted capability of the centralized value function. Conversely, due to the absence of a need to represent negative correlations between utility functions, the parameter search space for the joint cooperative policy exponentially decreases with the number of agents (Rashid et al., 2020c), facilitating a faster convergence of the model to appropriate cooperation during training (Hu et al., 2023; Rashid et al., 2020c).

4.2. MACE: contrastive exploration via value decomposition discrepancy

In this subsection, we introduce the MACE method proposed in this paper. Overall, MACE consists of four main components: the agent policy network, the first estimation part of fully expressive joint value function Q_{ji} , the second estimation part of the restricted expressive joint value function Q_{tot} , and the action preference network, as illustrated in Fig. 2. We will proceed to provide a detailed explanation of each part in the following parts.

4.2.1. Policy network of agents

This policy network primarily encompasses the process of Q -value estimation and output, which focuses on the decision-making process of the agents. It utilizes a GRU-based recurrent neural network, which processes the observation o and the previous action u_{t-1} of agents as inputs. After encoding through a feedforward neural network, the agent outputs its state-action value function Q_i from the hidden layer. The agent then employs a customized ϵ -greedy mechanism, which is driven by the action preference network, to select the cooperative action corresponding to the highest value, which is subsequently fed into the two distinct joint value function estimation parts for the calculation of VD discrepancy, as illustrated in part (a) of Fig. 2.

4.2.2. The first estimation part with fully expressive joint value function

The primary function of this module is to take the Q -value estimates from individual agents as input and produce an estimate of the centralized value function unconstrained by monotonicity. To enhance the representation of the collaborative policy of agents, this module leverages the self-attention mechanism to incorporate global state information to complement the utility functions and generate enhanced utilities $[\hat{Q}_i(\tau^i, u^i)]_{i=1}^n$. Through the use of soft attention, the module can selectively focus on and assimilate important data from the global state and surrounding agents, improving the policy network's learning of effective decision-making processes.

Once this enhanced utility functions $[Q_i(\tau^i, u^i)]_{i=1}^n$ are obtained, they are fed into a feedforward neural network for information encoding, and a state function $V(s)$ encoded with global state information is added to finally produce the centralized value function estimate Q_{ji} . Since no parameter constraints are imposed on the network in this module, there are no expressivity limitations between Q_{ji} and the utility function $[Q_i(\tau^i, u^i)]_{i=1}^n$. In other words, Q_{ji} has the capability to fit any function distribution, which can serve as a potential guiding direction for policy exploration. Therefore, the calculation process of Q_{ji} is described by the following Eq. (6):

$$\begin{aligned}
 Q_{ji}(\tau, \mathbf{u}) &= \text{Attention}(W_Q \cdot s, W_K \cdot s, [Q_i(\tau^i, u^i)]_{i=1}^n) + V(s) \\
 &= \text{softmax}\left(\frac{(W_Q \cdot s)(W_K \cdot s)^T}{\sqrt{d_k}}\right) \cdot [Q_i(\tau^i, u^i)]_{i=1}^n + V(s) \\
 &= [\hat{Q}_i(\tau^i, u^i)]_{i=1}^n + V(s),
 \end{aligned} \tag{6}$$

where W_Q and W_K denote the parameters of the Q -network and K -network used to encode the state s in the self-attention mechanism, d_k refers to the dimension of these encoded vector, and $\text{softmax}(\cdot)$ is applied

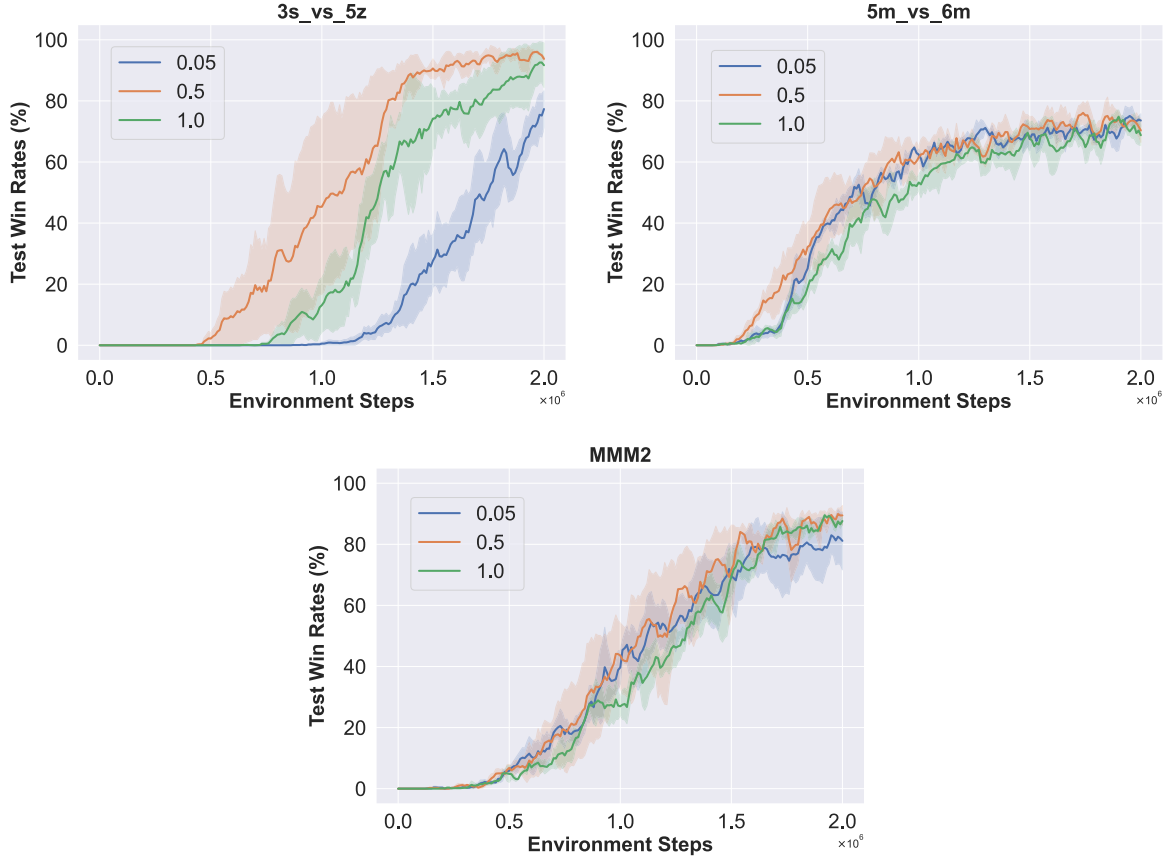


Fig. 14. The effect of parameter α in MACE along with training process.

to derive the scoring function. This process corresponds to part (b) of Fig. 2.

4.2.3. The second estimation part with restricted expressive joint value function

This part delivers the estimation for the second type of VD approach, which is illustrated in part (c) of Fig. 2. The central concept aligns with conventional VD techniques, leveraging a constrained mixing network (which forces the generated neural network weights to be positive to ensure a positive correlation between input and output) to combine the agents' utility functions, thereby yielding a centralized value function that can potentially decompose the cooperative tasks. This constrained neural network parameter compresses the action space during agent collaboration, allowing it to more quickly find suitable joint actions under batch sample training. However, due to the enforced positive constraint, the optimal joint action may not lie within this compressed action space, causing such VD methods to fail to converge to the optimal joint action, necessitating alternative policies of agents for exploration and guidance.

To clarify the impact of individual actions on the global outcome as represented by the utility function, MACE first decomposes each state-action value function $[Q_i(\tau^i, u^i)]_{i=1}^n$ into the state function $[V_i(\tau^i)]_{i=1}^n = \max[Q_i(\tau^i, u^i)]_{i=1}^n$ and advantage function $[A_i(\tau^i, u^i)]_{i=1}^n$ of each agent, which is represented by:

$$[Q_i(\tau^i, u^i)]_{i=1}^n = [A_i(\tau^i, u^i)]_{i=1}^n + [V_i(\tau^i)]_{i=1}^n \quad (7)$$

To incorporate the effect of global state information on the set of individual actions, this module still utilizes an attention mechanism to merge the global state information. To ensure compliance with IGM and preserve the positive correlation between the input and output of this mixing network, an absolute value function $\text{ABS}\{\cdot\} = |\cdot|$ is applied to the feature embedding part of the state and joint action. Consequently, the expression for the centralized value function estimation in this module is normalized as:

$$\begin{aligned} Q_{tot}(\tau, \mathbf{u}) &= A_{tot}(\tau, \mathbf{u}) + V_{tot}(\tau) \\ &= \text{Attention}(W'_Q \cdot s, W'_K \cdot \mathbf{u}, [A_i(\tau^i, u^i)]_{i=1}^n) + \sum_{i=1}^n V_i(\tau^i) \\ &= \text{softmax} \left(\frac{\text{ABS}\left\{ \left(W'_Q \cdot s \right) \left(W'_K \cdot \mathbf{u} \right)^T \right\}}{\sqrt{d'_k}} \right) \cdot [A_i(\tau^i, u^i)]_{i=1}^n + \sum_{i=1}^n V_i(\tau^i) \quad (8) \\ &= \text{softmax} \left(\frac{\left| \left(W'_Q \cdot s \right) \left(W'_K \cdot \hat{\mathbf{u}} \right)^T \right|}{\sqrt{d'_k}} \right) \cdot [A_i(\tau^i, u^i)]_{i=1}^n + \sum_{i=1}^n Q_i(\tau^i, u^i) \end{aligned}$$

Under such Q_{jt} and Q_{tot} settings, MACE still satisfies the IGM condition. Therefore, we have the following Theorem 1:

Theorem 1. An unrestricted and factorizable joint action-value function $Q_{jt}(\tau, \mathbf{u})$ is factorized by the individual $[Q_i(\tau^i, u^i)]_{i=1}^n$, if $Q_{tot}(\tau, \mathbf{u}) = V_{tot}(\tau) + A_{tot}(\tau, \mathbf{u})$, with $V_{tot}(\tau) = \sum_{i=1}^n V_i(\tau^i) = \sum_{i=1}^n \max Q_i(\tau^i, u^i)$ and $A_{tot}(\tau, \mathbf{u}) = \sum_{i=1}^n \rho_i(\tau^i, u^i) A_i(\tau^i, u^i)$, where $\rho_i(\tau^i, u^i) > 0$.

The proof process of Theorem 1 is presented in Appendix A.1.

4.2.4. Intrinsic reward calculation through value decomposition discrepancy

Based on the two distinct joint value function estimates Q_{jt} and Q_{tot} , MACE computes the intrinsic reward by evaluating the discrepancy between these two estimates. The underlying rationale is that the unrestricted joint value function Q_{jt} can provide a more accurate assessment of the expected returns for specific joint actions, while the restricted joint value function Q_{tot} can ensure the consistency of the joint actions and the set of individual optimal action, which is also depicted as the IGM principle. By quantifying the difference between these two estimates, MACE can identify actions that are potentially undervalued by Q_{tot} but are deemed favorable by Q_{jt} . This discrepancy serves as a contrastive exploration signal, encouraging agents to explore actions that may lead to better cooperative outcomes. Then the intrinsic reward is calculated as follows:

$$\text{IntrinT} = Q_{jt}(\tau, \mathbf{u}) - Q_{tot}(\tau, \mathbf{u}) \quad (9)$$

Actually, the value decomposition discrepancy is calculated as the difference between the two different types of centralized value functions, with a scaling factor ω to adjust its influence on the overall reward signal during training. Therefore, the final reward signal used for updating the agents' policies is a combination of the extrinsic reward from the environment and the intrinsic reward derived from the value decomposition discrepancy. In fact, such intrinsic reward can construct an equivalent Markov Decision Process with a tighter upper bound, thus accelerating the convergence of the collaborative policy trained by Q_{tot} . Therefore, we have the following [Theorem 2](#):

Theorem 2. We define Q_{tot} 's update operator $\mathcal{T} : \mathbb{R}^{|S| \times |U|} \rightarrow \mathbb{R}^{|S| \times |U|}$ as:

$$\mathcal{T}Q_{tot}(s, \mathbf{u}) := r(s, \mathbf{u}) + \omega(Q_{jt}(s, \mathbf{u}) - Q_{tot}(s, \mathbf{u})) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot|s, \mathbf{u})} \left[\max_{\mathbf{u}'} Q_{tot}(s', \mathbf{u}') \right]. \quad (10)$$

then we have

- $\mathcal{T}$ is a contraction mapping under the infinite norm $\|\cdot\|_\infty$, with a discounted factor of $\tilde{\gamma} = \frac{\gamma}{1+\omega} < \gamma < 1$.
- There exists a unique fixed point $Q_{tot|\omega}^*(s, \mathbf{u})$ that satisfies $\mathcal{T}Q_{tot|\omega}^*(s, \mathbf{u}) = Q_{tot|\omega}^*(s, \mathbf{u})$.
- When $Q_{jt}(s, \mathbf{u}) = Q^*(s, \mathbf{u})$, this fixed point is the true optimal Q function: $Q_{tot|\omega}^*(s, \mathbf{u}) = Q^*(s, \mathbf{u})$.
- In general, the approximation error satisfies:

$$\|Q_{tot|\omega}^*(s, \mathbf{u}) - Q^*(s, \mathbf{u})\|_\infty \leq \frac{\omega}{1+\omega-\gamma} \|Q_{jt}(s, \mathbf{u}) - Q^*(s, \mathbf{u})\|_\infty. \quad (11)$$

In particular, if $\|Q_{jt}(s, \mathbf{u}) - Q^*(s, \mathbf{u})\|_\infty \leq \sigma$, then $\|Q_{tot|\omega}^*(s, \mathbf{u}) - Q^*(s, \mathbf{u})\|_\infty \leq \frac{\omega\sigma}{1+\omega-\gamma}$.

The proof process of [Theorem 2](#) is presented in [Appendix A.2](#).

4.2.5. Action preference network

Beyond leveraging discrepancies in VD methods to guide implicit exploration, MACE incorporates an action preference network to learn agents' cooperative preferences, as illustrated in part (a) of [Fig. 2](#). Specifically, each agent's decision network has two output components: Q -function estimation and the action-value distribution. These two components share a preceding recurrent network encoding, as shown in [Fig. 3](#). During cooperative training, once agents infer their action preferences, this network encourages sampling actions based on these preferences rather than following a greedy approach, fostering prior cooperative exploration. Each transition from agent-environment interaction updates the action preference network in MACE, while other model parameters are updated using samples from the replay buffer. This approach modifies the vanilla ϵ -greedy action-sampling mechanism, referred to here as customized ϵ -greedy.

Unlike traditional value-based reinforcement learning methods, the action preference network introduces an additional neural network module that learns to predict the exploration preferences of agents based

on their historical action choices and policy entropy. It does not directly determine the optimal action (i.e., the action with the largest Q value) or other actions in the current interaction process with a fixed probability. Instead, it evaluates the advantage of each action of the agent through a neural network $\eta_\phi(\cdot)$, and then normalizes it using the Softmax function to obtain the advantage probability of each action. Furthermore, such a dominant probability is added to the original ϵ -greedy probability, thereby indirectly exploring the actions of the agent with a larger Q value, which is depicted as:

$$\pi_{\text{custom-}\epsilon}(u^i | o^i) = \begin{cases} 1 - \epsilon + \epsilon \cdot \text{Softmax}(\eta_\phi(u^i | o^i)), & \text{if } u^i = u_*^i \\ \epsilon \cdot \text{Softmax}(\eta_\phi(u^i | o^i)), & u^i \neq u_*^i \end{cases} \quad (12)$$

where $u^i \sim \eta_\phi(\cdot | o^i)$ represents the preferred action sampled from the action preference network $\eta_\phi(\cdot | o^i)$ of individual agents in cooperating. The more preference of u^i , the higher the probability it will be chosen. It is noted that due to the normalization property of the Softmax function, [Eq. \(12\)](#) satisfies $\sum_{u^i \in U^i} \pi_{\text{custom-}\epsilon}(u^i | o^i) = 1$ over the action set U^i , which means it exactly forms a valid policy distribution. Here, MACE uses the policy entropy of an individual agent to determine the update direction of its action preference network. In other words, we hope that the principle of the action selection distribution output by the preference network follows that the larger the Q value, the greater the probability of action selection. Therefore, we hope that the preference network $\eta_\phi(\cdot)$ can maximize the entropy of this policy. Since each agent under the CTDE architecture independently selects its own actions, we can maximize the sum of the policy entropies of all agents. The gradient of parameter update is represented as:

$$\Delta\phi \propto \sum_{i=1}^n \mathbb{E}_{o^i \sim \mathcal{P}, u^i \sim \eta} [(Q_i^n(o^i, u^i) - V^\eta(o^i)) \nabla_\phi \log \eta_\phi(u^i | o^i) + \beta \nabla_\phi \mathcal{H}^\eta(o^i)]. \quad (13)$$

where ϕ represents the value of the parameters of the action preference network, and $\mathcal{H}^\eta(o^i) = -\sum_u \eta_\phi(u^i | o^i) \log \eta_\phi(u^i | o^i)$ represents the Shannon entropy of $\phi(u^i | o^i)$ at observation o^i of agent i . In practice, we adopt the same adaptive temperature coefficient adjustment method as the Soft Actor-Critic algorithm ([Haarnoja et al., 2018b](#)). That is, after an initialized beta coefficient β and target policy entropy ξ_{entropy} were given, the beta coefficient was still updated according to this goal:

$$\min_{\beta \geq 0} \max_{\phi} \sum_{i=1}^n \left(\left(\mathbb{E}_{o^i \sim \mathcal{P}, u^i \sim \eta} [A_i^n(o^i, u^i) \eta_\phi(u^i | o^i) + \beta \mathcal{H}^\eta(o^i)] \right) - \beta \xi_{\text{entropy}} \right) \quad (14)$$

and then with loss function $\mathcal{L}(\beta)$:

$$\mathcal{L}(\beta) = \sum_{i=1}^n \mathbb{E}_{o^i \sim \mathcal{P}, u^i \sim \eta} [-\beta \log \eta_\phi(u^i | o^i) - \beta \xi_{\text{entropy}}] \quad (15)$$

Still, the objection function for the action preference network can be written as:

$$\begin{aligned} \mathcal{L}(\phi) &= \sum_{i=1}^n \mathbb{E}_{o^i \sim \mathcal{P}, u^i \sim \eta} [(Q_i^n(o^i, u^i) - V^\eta(o^i)) \eta_\phi(u^i | o^i) + \beta \mathcal{H}^\eta(o^i)] \\ &= \sum_{i=1}^n \mathbb{E}_{o^i \sim \mathcal{P}, u^i \sim \eta} [A_i^n(o^i, u^i) \eta_\phi(u^i | o^i) + \beta \mathcal{H}^\eta(o^i)] \end{aligned} \quad (16)$$

where $\mathcal{H}^\eta(\cdot)$ is the policy entropy of agent i , and $\xi_{\text{entropy}} = -\dim(U^i)$ is the target entropy value.

4.3. The implementation of MACE

In this subsection, the implementation of MACE is introduced in detail and also described in [Algorithm 1](#). Similar to traditional cooperative MARL, MACE also updates the parameters of the centralized value function network using the team rewards obtained through interactions between the module and the environment. However, MACE differs in

that it incorporates a novel action preference network and utilizes its output to adjust the ϵ -greedy mechanism to enhance agents' action preferences. Furthermore, MACE adaptively computes the intrinsic target at each step based on the discrepancy between Q_{jt} and Q_{tot} , which implicitly modifies the centralized value function target for cooperative actions to promote broader action exploration.

Algorithm 1 MACE: multi-agent contrastive exploration.

```

Initialize parameters of networks  $\theta$  and their targets  $\theta^-$  with  $\theta^- \leftarrow \theta$ ;
Initialize parameters of action preference network  $\phi$  and its target  $\phi^-$  with  $\phi^- \leftarrow \phi$ ;
Initialize entropy parameters:  $\beta, \xi_{\text{entropy}}$ ;
Initialize learning rate  $\lambda_{vd}, \lambda_\eta, \lambda_\beta$ , replay buffer  $\mathcal{D}$ , target network update interval  $\mathcal{K}_v$ , action preference update interval  $\mathcal{K}_{pref}$ , and set step = 0;
while step < stepmax do
   $t = 0, s_0 = \text{initial state}$ ;
  while  $s_t \neq \text{terminal}$  and  $t < \text{episode limit}$  do
    for each agent  $i$  do
       $\tau_t^i = \tau_{t-1}^i \cup \{(o_t^i, u_{t-1}^i)\}$ ;
      Select action  $u_t^i$  with  $\pi_{\text{custom-}\epsilon}(u_t^i | o_t^i)$ ;
    end for
    Get reward  $\mathbf{u}_t, r_t$  and next state  $s_{t+1}$ ;
     $\mathcal{D} = \mathcal{D} \cup \{(s_t, \mathbf{u}_t, r_t, s_{t+1})\}$ ;  $t = t + 1, \text{step} = \text{step} + 1$ ;
    if step %  $\mathcal{K}_{pref} = 0$  then
      Compute loss function of action preference network  $\mathcal{L}(\phi)$ ;
      Update action preference network:  $\phi \leftarrow \phi - \lambda_\eta \nabla_\phi \mathcal{L}_\eta(\phi)$ ;
      Update temperature parameter:  $\beta \leftarrow \beta - \lambda_\beta \nabla_\beta \mathcal{L}_\beta$ ;
    end if
  end while
  if  $|\mathcal{D}| > \text{batch-size}$  then
     $b \leftarrow \text{Select random batch of episodes from } \mathcal{D}$ ;
    for  $t$  in each trajectory in batch  $b$  do
      Estimate  $Q_{jt}$  with (6) and  $Q_{tot}$  with (8);
      Estimate their respective target values  $Q_{jt}^-$  and  $Q_{tot}^-$ ;
      Compute VD loss function  $\mathcal{L}_{vd}(\theta)$  with (18);
      Update the VD network:  $\theta \leftarrow \theta - \lambda_{vd} \nabla_\theta \mathcal{L}_{vd}(\theta)$ ;
    end for
  end if
  if step %  $\mathcal{K}_v = 0$  then
    Update the target network  $\theta^- \leftarrow \theta$ ;
  end if
end while

```

First, we initialize the network parameters of the policy networks of agents, the action preference network, and the two types of VD parts (including the parameters of their respective attention modules), as well as their target network parameters. Next, we set the parameters for experience replay buffer $\mathcal{D}$, learning rate η , the size of batch sampling batch_size and target network update_interval . We evaluate the utility functions $[Q_i(\tau^i, u^i)]_{i=1}^n$ with the individual observation o^i and last action u_{t-1}^i . Then we evaluate the Q_{jt} and Q_{tot} of the two parts of VD and calculate the discrepancy between these two centralized value function estimations to obtain the intrinsic target.

During the training, MACE extracts a small batch of interaction trajectories from $\mathcal{D}$ and estimates the utility function $[Q_i(\tau^i, u^i)]_{i=1}^n$ and these two centralized value function $\{Q_{jt}, Q_{tot}\}$. During this process, the action preference network is updated based on the trajectory of each interaction between the model and the environment. At this point, MACE calculates the average policy entropy of all agents using Eq. (16) and (13) to update the parameters of the action preference network, which adjusts the agents' vanilla ϵ -greedy selection mechanism and get the action preference policy based on this average entropy, ensuring the maximum entropy for the entire team's collaborative policy.

Still, an intrinsic target will be obtained by scaling the discrepancy between Q_{jt} and Q_{tot} with a factor ω to encourage the model to jump out of local optimal and easier to find optimal joint action, which is concluded as $\omega \cdot (Q_{jt} - Q_{tot})$. The intrinsic target in MACE leverages contrastive learning principles, interpreting the gap between Q_{jt} and Q_{tot} as a trainable metric to minimize. Ideally, when MACE achieves the joint optimal action, this difference becomes zero; for non-optimal actions, Q_{jt} 's full expressiveness makes it more suited than Q_{tot} to capture the optimal joint action's value function. Reducing this gap implicitly encourages Q_{tot} to escape local optima, thus indirectly boosting exploration - an approach aligned with contrastive learning's policy of maximizing separation between positive and negative samples.

On the other hand, MACE also adjusts parameter update weights based on the difference between Q_{jt} and Q_{tot} inspired by WQMIX (Rashid et al., 2020a). Since the constrained centralized value function often underestimates the impact of the optimal joint action, which makes it difficult for the model to find the optimal action, MACE sets the update weight of Q_{jt} to $\alpha = 1$, while setting the update weight of Q_{tot} to a value $\alpha < 1$ where its estimation is less than Q_{jt} in the same batch, thus emphasizing the importance of the optimal joint action, which is concluded as:

$$\alpha = \begin{cases} 1, & Q_{jt}(\tau, \mathbf{u}, s) > Q_{tot}(\tau, \mathbf{u}, s) \\ [0, 1), & Q_{jt}(\tau, \mathbf{u}, s) \leq Q_{tot}(\tau, \mathbf{u}, s) \end{cases} \quad (17)$$

Then we can update the parameters of all the networks expected for the action preference network just like the way in DQN (Mnih et al., 2015) through the back-propagation:

$$\begin{aligned} \mathcal{L}_{vd}(\theta) &= \mathcal{L}_{jt}(Q_{jt}, y_{jt}^-) + \mathcal{L}_{tot}(Q_{tot}, y_{tot}^-) \\ \mathcal{L}_{jt}(Q_{jt}, y_{jt}^-) &= (Q_{jt}(\tau, \mathbf{u}, s) - y_{jt}^-)^2 \\ \mathcal{L}_{tot}(Q_{tot}, y_{tot}^-) &= \alpha \cdot (Q_{tot}(\tau, \mathbf{u}, s) - y_{tot}^-)^2 \end{aligned} \quad (18)$$

where y_{jt}^- and y_{tot}^- are the target estimated values of Q_{jt} and Q_{tot} respectively, which is calculated as $y_{jt}^- \leftarrow r + \gamma \cdot \max_{s'} Q_{jt}(\tau', \mathbf{u}', s')$ and $y_{tot}^- \leftarrow \omega \cdot \text{intrinsic} + r + \gamma \cdot \max_{s'} Q_{tot}(\tau', \mathbf{u}', s') = \omega \cdot (Q_{jt}(\tau, \mathbf{u}, s) - Q_{tot}(\tau, \mathbf{u}, s)) + r + \gamma \cdot \max_{s'} Q_{tot}(\tau', \mathbf{u}', s')$. Then the overall loss function that the model needs to update is:

$$\mathcal{L} = \mathcal{L}_{vd}(\theta) + \mathcal{L}(\phi) + \mathcal{L}(\beta) \quad (19)$$

5. Experiments

In this section, we evaluate the effectiveness of the proposed MACE method. More concretely, we investigate whether the specially designed modules in MACE can enhance exploration and address challenges such as the *relative-overgeneralization* problem. Moreover, we analyze the roles of each module in MACE and demonstrate its advantages compared to various baseline approaches. We implement all the compared methods in the PyMARL framework (Samvelyan et al., 2019), and all the hyper-parameters will be **KEEP the SAME** as in PyMARL for the sake of fairness. The purpose of all the experiments and tests is to address the following questions:

- Question(1):** Does MACE effectively boost exploration and surpass other baseline methods in test performance?
- Question(2):** Which specific part of MACE is responsible for improving the performance and exploration?
- Question(3):** How does the action preference network within MACE impact the overall performance of the model?

We evaluate MACE and other baseline algorithms across various types and difficulty levels of test scenarios, including Matrix Game (Rashid et al., 2018) and StarCraft Multi-Agent Challenge (SMAC) environment (Samvelyan et al., 2019) to answer the above questions. These experimental tasks offer a diverse and challenging set of scenarios, aimed at demonstrating the algorithm's ability to handle the *relative-overgeneralization* issue (Wei et al., 2018). The subsequent sections de-

Table 2
Characteristics of the scenarios in SMAC experiments (Wan, 2023, 2024).

name	level	challenge
2s_vs_1sc	easy	alternating fire
2m_vs_1z	easy	alternating fire
1c3s5z	easy	heterogeneous & symmetric
bane_vs_bane	easy	positioning
5m_vs_6m	hard	heterogeneous & symmetric
3s_vs_5z	hard	heterogeneous & symmetric
2c_vs_64zg	hard	positioning
MMM2	super-hard	circuitous tactics
3s5z_vs_3s6z	super-hard	circuitous tactics

scribe the three test environments in detail, including model parameters, results, and ablation studies addressing the noted concerns.

5.1. Experimental scenarios

5.1.1. Matrix game

The matrix game is a simple mathematical model used to study the interaction and decision-making scenarios of two agents in a shared environment. In this setting, each agent has three possible actions, and their choices result in simultaneous outcomes according to a shared pay-off function, we consider two types of cases (near-degenerate case and stochastic case). This symmetric matrix game model exhibits an optimal joint action (A, A) and (C, C), where agents possess an observation with all values set to 1, indicating that agents solely consider interactive actions to impact specific cooperative rewards. Agents can only choose one action per round of interaction. The purpose of agent collaboration is to find the optimal joint action as soon as possible and give it the correct estimation. This model allows for a rapid evaluation of whether the tested algorithms satisfy the IGM condition and whether the centralized value function under the joint actions of agents can accurately estimate the value functions.

5.1.2. SMAC

StarCraft II, being a real-time strategy game, presents an excellent opportunity to address cooperative challenges in the multi-agent domain. SMAC (Samvelyan et al., 2019) utilizes Blizzard's StarCraft II Machine Learning API and DeepMind's PySC2 as an interface to engage with the game environment, as the sparring scenarios are shown in Fig. 4. Our focus lies on the micromanagement challenges within SMAC, where each unit is under the control of an independent agent, relying solely on limited local observation. These unit groups must be trained to engage an opposing army under the centralized control of the game's built-in scripted AI. The objective of cooperative allies is to maximize damage dealt to enemy units while minimizing received damage. Symmetrically placed opposing groups with varied initial unit placements characterize the scenario across episodes. The characteristics of the testing scenarios are summarized in Table 2.

5.2. Environmental setup

To ensure consistency between the comparative methods, we implement all these algorithms within the PyMARL platform and standardize the network architecture and hyperparameters in a fair way. Following PyMARL's specifications, the agent architecture utilizes a DRQN with a 64-dimensional hidden state in its recurrent layer. In most of the SMAC scenarios, each agent independently adopts ϵ -greedy action selection based on its Q_i , where ϵ decays from 1.0 to 0.05 over 50,000 time steps (except for the super-hard scenario 3s5z_vs_3s6z, which is annealed in 1,000,000 time steps) and then remains constant thereafter. As for The Matrix Game, all agents adhere to full exploration, i.e., $\epsilon = 1$, and remain unchanged during training. A fixed discount factor $\gamma = 0.99$ is maintained across experiments. RMSprop with $\alpha = 0.99$

Table 3
Pay-off matrix of the one-step game (near-degenerate case). Boldface means optimal/greedy actions from the state-action value.

(a) Payoff of Matrix Game			
$a_1 \backslash a_2$	A	B	C
A	20	-12	-12
B	-12	0	0
C	-12	0	19.9
(b) Estimation of MACE			
$a_1 \backslash a_2$	A	B	C
A	20.31	-11.71	-11.32
B	-11.37	-0.18	-0.07
C	-12.04	0.03	19.57
(c) Estimation of QMIX			
$a_1 \backslash a_2$	A	B	C
A	-5.06	-5.00	-4.97
B	-4.97	-0.12	0.36
C	-5.04	0.29	20.12
(d) Estimation of QPLEX			
$a_1 \backslash a_2$	A	B	C
A	19.59	-12.45	-12.74
B	-11.87	-0.33	-0.04
C	-17.25	0.26	19.07
(e) Estimation of WQMIX			
$a_1 \backslash a_2$	A	B	C
A	19.84	-12.21	-12.09
B	-12.39	-0.21	-0.23
C	-12.45	-0.24	19.85
(f) Estimation of VDN			
$a_1 \backslash a_2$	A	B	C
A	-2.17	-4.68	1.98
B	-4.64	-7.19	-0.51
C	2.23	-0.30	6.37

is implemented, excluding weight decay and momentum. The training learning rate is consistently set at 5×10^{-4} for all SMAC experiments.¹

For the constrained VD part, the feedforward neural network encoding dimension is set to 32, with a corresponding weight-generating hypernetwork consisting of 2 hidden layers, each with 64 neurons. The unconstrained VD part has an encoding dimension of 256. The attention modules within both networks have hidden layer dimensions of 64 neurons. The scaling factor for the intrinsic target is set to $\omega = 0.01$, and the weight factor for Q_{tot} updates is set to $\alpha = 0.5$. For the action preference network, MACE utilizes two feedforward networks with 64 neurons each following the encoding network's output to produce outputs for action distribution and Q-function estimation. The learning rate of the action preference network is set to 0.0005, with the parameters update occurring every 50 time steps to ensure stability of the whole training. Additionally, a temperature coefficient $\beta = 0.1$ is applied to the agent policy entropy when updating the action preference network.

The training pauses at intervals of 10,000-time steps to conduct 16 evaluation episodes. The replay buffer retains the latest 5000 episodes, from which 32-episode batches are uniformly sampled for training. All

¹ We use the SC2.4.10 version of SMAC throughout all the testing scenarios.

Table 4
Pay-off matrix of the one-step game (stochastic case).

(a) Payoff of Matrix Game				
$a_1 \backslash a_2$	A	B	C	
A	7.2	-1.3	-12	
B	-4.2	0	3.8	
C	-17	-2.5	10	
(b) Estimation of MACE				
$a_1 \backslash a_2$	A	B	C	
A	6.93	-1.50	-11.81	
B	-4.25	-0.07	3.80	
C	-17.14	-2.65	9.88	
(c) Estimation of QMIX				
$a_1 \backslash a_2$	A	B	C	
A	-6.29	-3.69	-1.21	
B	-4.57	0.55	4.80	
C	-4.67	-0.19	4.09	
(d) Estimation of QPLEX				
$a_1 \backslash a_2$	A	B	C	
A	6.78	-1.42	-11.71	
B	-3.34	0.25	3.70	
C	-17.62	-2.69	9.7	
(e) Estimation of WQMIX				
$a_1 \backslash a_2$	A	B	C	
A	7.30	-1.33	-11.93	
B	-4.23	0.06	3.87	
C	-16.59	-2.75	10.28	
(f) Estimation of VDN				
$a_1 \backslash a_2$	A	B	C	
A	-4.35	-1.05	1.26	
B	-2.97	0.21	2.50	
C	-6.02	-2.88	-0.56	

agent networks share the same parameters, and target networks are updated every 200 episodes. To ensure performance comparisons remain fair, all baseline parameters match their respective settings in official code bases.

5.3. Main results

This subsection's primary aim is to address **Question 1**. To this end, we utilize several VD approaches as benchmarks to assess the properties of MACE within the different difficult levels of Matrix Game. As for SMAC, we consider 9 combat scenarios, from **easy** (2m_vs_1z, 2s_vs_1sc, 1c3s5z, bane_vs_bane), **hard** (5m_vs_6m, 3s_vs_5z, 2c_vs_64zg) and **super hard** (3s5z_vs_3s6z, MMM2). Consistent with prior research (Hu et al., 2023; Rashid et al., 2020a, 2018; Wang et al., 2020), the AI's game difficulty is set to **Very Hard** under the standard configuration to create a highly challenging cooperative context. We present the training results separately for each of these three environments.

5.3.1. Evaluated results in matrix game

First, we conduct the experiments with MACE and other baselines in matrix games, as shown in Table 3 and Table 4. Table 3 and Table 4 represent matrix games with different types. From the results in Table 3, we observe that MACE is able to effectively learn the joint optimal action,

with a minimal difference between the estimated centralized value and the true values. WQMIX ultimately converges to the suboptimal joint action, which, despite its theoretical guarantees, might require more training time. QPLEX also learns the correct cooperative policy, but its learning process exhibits large fluctuations, and in the end, its centralized value function estimation deviates more from the true value than MACE. QMIX and VDN cannot find the correct joint actions. Then in the stochastic case, as shown in Table 4, MACE is also able to converge to the true optimal joint actions with high accuracy in centralized value functions. WQMIX and QPLEX can find the joint optimal actions, but QMIX and VDN still fail to explore the reward distribution with significant deviations in their estimated Q_{jt} functions from the true values. These results further validate the effectiveness of MACE in accurately estimating centralized value functions and learning optimal cooperative policies.

We then show the discrepancy histogram of MAE in Fig. 5 and Fig. 6. We calculate the discrepancy value defined by $|Q_{jt} - Q_{tot}|$. As the training progresses, the discrepancy value of the optimal joint action gradually decreases. This is because Q_{jt} has learned the correct estimation of optimal joint action, effectively guiding the update of Q_{jt} and indirectly confirming the convergence of MACE and its satisfaction of the IGM condition. The difference between the histograms of these two cases lies in that in the near-degenerate case, although the model makes a relatively accurate estimation of the joint optimal action at the beginning, Q_{tot} still wrongly estimates the non-optimal action, thus resulting in a relatively large discrepancy value. As the model is trained, the intrinsic rewards brought about by this discrepancy will cause fluctuations in the function estimation. MACE continues to train the cooperative policy based on the adjusted intrinsic reward until all the joint actions are close to being correctly estimated, at which point the discrepancy values converge to a relatively small value. In the case of the stochastic case, Q_{tot} might have made a relatively good valuation calculation from the training beginning, so the entire process went very smoothly. It can be seen that as the training progresses, the discrepancy values gradually decrease and converge. This process illustrates the changes in the value decomposition discrepancy and can clearly demonstrate the role process of the intrinsic reward it constructs in training. The contrastive nature of the differential distribution further validates the effectiveness of the MACE method in addressing the coordination challenges in multi-agent settings.

5.3.2. Evaluated results in SMAC

The training results of all the comparative approaches in SMAC are illustrated in Fig. 7. It shows that MACE almost outperforms all the other baseline methods in nearly all scenarios as the model training progresses. Notably, MACE achieved unexpected results in two **super hard** scenarios. In MMM2, MACE consistently became the best and outperformed ResQ, which also applied the $Q(\lambda)$ method to smooth the obtained reward information, a technique that has been widely proven to be highly effective in RL model training. While in 3s5z_vs_3s6z, MACE also demonstrated clear advantages. After 2 million training steps (in most other methods, the model's win rate can only be observed and differentiated after 5 million training steps (Wan, 2024; Zhao et al., 2024b)), other methods were almost unable to achieve any win rates, while MACE steadily improved and ultimately achieved a median win rate of nearly 15%, showcasing the superiority of this proposed method. Even in some relatively simple scenarios (2m_vs_1z, 2s_vs_1sc, bane_vs_bane), MACE converges more quickly than other methods, with minimal fluctuations, demonstrating the superiority of this method in training collaborative policies after integrating different types of VD estimates. This indicates that the introduction of value decomposition discrepancy and exploration preference network can indeed help the collaborative policy achieve faster convergence and explore more useful states. On the other hand, some enhanced exploration methods, such as MAVEN, did not perform well. The reason for this might lie in the fact that MAVEN was trained using multiple parallel environments for

sample collection. The faster sample collection process slowed down the difference between the current update policy of the model and the policy used when generating samples, then reduced the impact of model errors on training. These results underscore the effectiveness of MACE in SMAC scenarios and its potential to be a valuable tool in the development of more sophisticated RL algorithms.

Besides, we provide the multi-seed error bars of MACE and other baselines in 3s_vs_5z and MMM2 for a more comprehensive comparison, as shown in Fig. 8. The solid bar chart represents the average testing win rate, and the wireframe above it represents the fluctuation range under multiple seed results. We can conclude that in these more complex heterogeneous agents or scenarios that require precise cooperation (3s_vs_5z, MMM2), MACE has demonstrated its superiority. Its performance can be improved by these baseline methods and gradually increases with training performance, verifying the effectiveness of the additional module proposed in the manuscript.

5.4. Ablation studies

In this subsection, we conduct ablation studies on the SMAC environments of 3s_vs_5z, 5m_vs_6m, and MMM2. These environments cover various difficulty levels and types of cooperative abilities, allowing us to thoroughly investigate the impact of various components of MACE. The primary aim of these experiments is to identify which elements of the additional techniques within MACE contribute most to enhancing the overall model's performance, answering **Question 2**. Furthermore, we explore different update frequencies for the action preference network, illustrating the significant role of an appropriate update frequency in MACE, which addresses **Question 3**. Then, we also test the influence of different parameters in the proposed modules in MACE on the model and analyze the specific effect of their magnitude on performance. Fi-

nally, we test the performance of MACE on the sparse reward setting to detect its generalization ability.

5.4.1. The influence of the additional optimism on the MACE in the model training

The experimental content in this subsection is primarily designed to investigate the impact of incorporating optimism into the MACE model on training performance. The results of the experiments are presented in Fig. 9. Here, (*MACE w optimism*) indicates the MACE model, while (*MACE w.o optimism*) indicates that without the optimism part. We can conclude that the MACE model with optimism performs better than its opponent, which mainly manifests in accelerating the policy convergence about 5%-18% in 3s_vs_5z and 1%-8% in MMM2 during training, allowing the collaborative model to achieve task success more quickly. Another possible explanation is that the optimism component, constructed from the information discrepancy between two different types of VD methods, represents the difference between the restricted and unrestricted centralized value functions. The additional optimism allows the restricted VD component to continuously reduce this difference during the update process, enabling the model to escape from local optima and fit the true value function distribution more quickly, thus improving the overall model performance. In other words, MACE can perceive features from the discrepancy between these two VD estimates that are more conducive to model updates during the training process, thus enhancing the overall training of the model. However, in 5m_vs_6m, the optimism part has no obvious improvement effect on the model. We believe the reason is that this cooperative scenario mainly tests the cooperative ability of homogeneous agents. Due to the homogeneity of agent attributes, the observation vectors of each agent are relatively similar, which may result in smaller changes in the state representation after

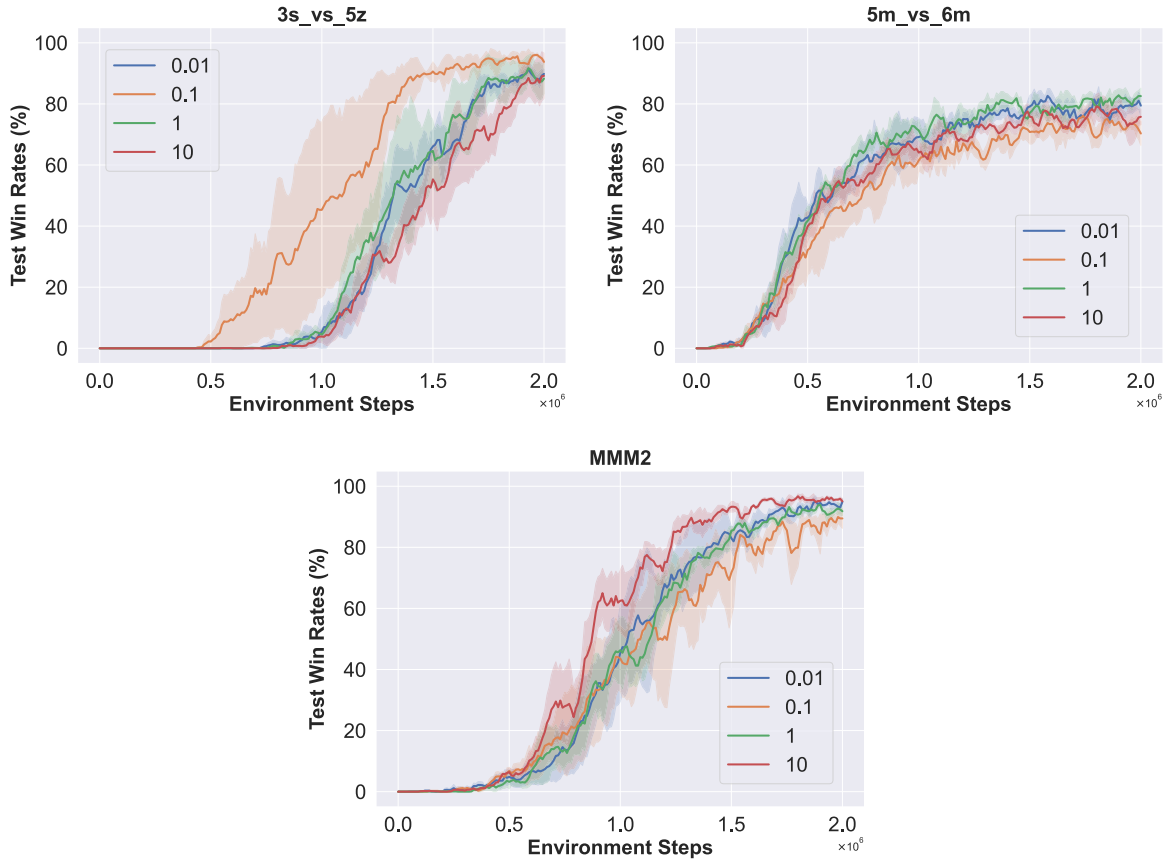


Fig. 15. The effect of parameter β in MACE along with training process.

exploration, thus making it difficult to effectively reflect the role of this module.

5.4.2. The analysis of action preference network of MACE in the process of training

The main purpose of this ablation study is to analyze whether the action preference network, which is associated with the average policy entropy of the agents and added in MACE, has a possible effect on the training of the cooperative policy. We recorded the results along with the training in Fig. 10. Here, (*MACE w preference*) represents the whole MACE model, while (*MACE w.o preference*) represents that method without the updating of the action preference network, which maintains the ϵ -greedy action selection mechanism. The impact of the action preference network manifests differently in various scenarios. As the previous subsection, due to the similar characteristics of homogeneous agents, the additional exploration introduced by the action preference network can only provide limited improvement in 5m_vs_6m. While in 3s_vs_5z, the action preference network still improves the learning speed of the model and ultimately converges to a collaborative policy with nearly 100% win rate, which many other baseline methods fail to achieve. In the **Super-Hard** scenario MMM2, the action preference network can improve the model's final performance by approximately 8%. A potential explanation is that in more difficult scenarios, agents need to adapt their policies more quickly to train collaborative training. The action preference network continuously adjusts the action selection mechanism based on the principle of maximizing the average policy entropy, enabling agents to conduct further exploration and make more task-specific collaborative action choices, thus improving the model's final performance.

5.4.3. The analysis of the additional exploration modules in MACE

To verify the overall effect improvement brought by these two modules, we also conducted another ablation experiment, that is, deleting the action preference network and VD discrepancy-based intrinsic reward modules simultaneously. As shown in Fig. 11, (*MACE*) indicates the original MACE model, while (*MACE w.o optimism & preference*) indicates that without any additional exploration part. We find that in the three tested scenarios, after removing all the exploration modules, the performance of MACE declines to varying degrees. Without the assistance of the exploration module in MACE, the model is inferior both in terms of policy training efficiency and the final results. Especially in the scenario of heterogeneous confrontation (3s_vs_5z, MMM2), the winning rate gap of the cooperative policy of agents can reach 12%-25%. This fully demonstrates that the additional exploration brought by the intrinsic reward and action preference network in MACE can effectively help the model accelerate training and have better performance. Even in the homogeneous adversarial scenario 5m_vs_6m, MACE with additional exploration modules has certain advantages and can achieve an improvement of approximately 3%-5% during the training process. This ablation experiment can verify the performance improvement brought by the additional exploration to the model.

5.4.4. The analysis of the update frequency of action preference network in MACE

The purpose of this subsection is to examine the effect of the update frequency of the action preference network in the MACE model on the training of collaborative policies. Since the update interval of the target network is 200, we set the update intervals of the action preference network to (10, 50, 100, 200) to test the model's collaborative ability. The training performance is recorded in Fig. 12, which can be concluded that the impact of the action preference network update frequency on performance is minor in easy and hard level scenarios such as 3s_vs_5z and 5m_vs_6m, only affecting the convergence speed of some policies. However, in a more complex MMM2 scenario, the update frequency has a significant effect on the final performance. This could be because the agents encounter new observations and action features

with considerable variation during each interaction with the environment. When the action preference network updates too quickly, it may cause instability in the agent's policy entropy, leading to overly random action selection, which negatively impacts the training of cooperative policies, especially when there is more uncertainty in action selection during the early stages of training. On the other hand, when the action preference network updates more slowly, it fails to keep pace with the policy learning process, potentially causing the agent to make incorrect action choices and hindering the collaborative process. Therefore, we set the update frequency of the action preference network between 50 and 100 in the model (which is set to 50 across all the experiments), as it strikes a balance and works well across different training scenarios.

5.4.5. The analysis of the scaling factor ω of intrinsic reward in MACE

For experiments on the additional hyperparameters, we first examine the impact of the intrinsic reward scaling coefficient $\omega \in \{0.001, 0.01, 1, 100\}$ on multi-agent learning across three scenarios: 3s_vs_5z, 5m_vs_6m and MMM2. The testing results reveal a strong task-dependent sensitivity to ω , which is shown in Fig. 13. In 3s_vs_5z, $\omega = 0.01$ achieves optimal performance (up to 90% win rate) and fastest convergence; higher values ($\omega = 1, 100$) severely degrade performance, confirming that excessive intrinsic rewards disrupt learning by masking extrinsic signals. In MMM2, both $\omega = 0.01$ and $\omega = 1$ perform robustly (over 85% win rate), indicating greater tolerance for stronger auxiliary rewards. Conversely, in the 5m_vs_6m scenario, $\omega = 0.001$ yields the highest final performance (about 85%), suggesting that subtle intrinsic guidance is preferable for intricate symmetrical coordination tasks. Overall, the optimal value of ω is task-specific, we choose $\omega = 0.01$ to ensure relatively good results in all scenarios and balance the stability of the exploration incentive policy.

5.4.6. The analysis of weighting factor α on non-optimal joint actions

Next, we evaluate the sensitivity of α , which governs the contribution of suboptimal joint actions to the Q_{tot} update. In other words, the function of α is the same as the parameter w in WQMIX. Here we consider $\alpha \in \{0.05, 0.5, 1.0\}$ to explore its influence on MACE, with the results shown in Fig. 14. Across three benchmark scenarios (3s_vs_5z, 5m_vs_6m, and MMM2), α is shown to critically influence both convergence dynamics and final performance. In 3s_vs_5z, $\alpha = 0.5$ yields optimal performance, while $\alpha = 0.05$ results in slow convergence and sub-par asymptotic performance, indicating insufficient guidance for Q_{tot} estimation. Although $\alpha = 1$ also has certain effects, it does not reduce the update weight of non-optimal actions, resulting in the fact that for Q_{tot} , the optimal and non-optimal actions are of equal importance. This makes the collaborative policy lack a clear guiding direction during the update process. For the 5m_vs_6m and MMM2 tasks, $\alpha = 0.5$ consistently achieves the highest or near-optimal win rates (over 85%), outperforming both extreme values. This demonstrates that $\alpha = 0.5$ strikes an effective balance: it leverages information from suboptimal trajectories to stabilize Q_{tot} learning without diluting the gradient signal for the optimal policy. Consequently, $\alpha = 0.5$ emerges as a robust and empirically optimal default setting, significantly outperforming its extreme counterparts.

5.4.7. The analysis of the temperature β in policy entropy of action preference network

Then we investigate the impact of the entropy temperature coefficient $\beta \in \{0.01, 0.1, 1, 10\}$, which is analogous to the α in Soft Actor-Critic. β governs the magnitude of the influence brought by policy entropy by weighting the entropy term in the policy objective. The results in Fig. 15 across three scenarios (3s_vs_5z, 5m_vs_6m, MMM2) reveal a strong task-specific sensitivity. In 3s_vs_5z, $\beta = 0.1$ achieves optimal performance (over 90% win rate) with rapid convergence; $\beta = 0.01$ converges slowly, while $\beta = 1$ and $\beta = 10$ cause severe degradation,

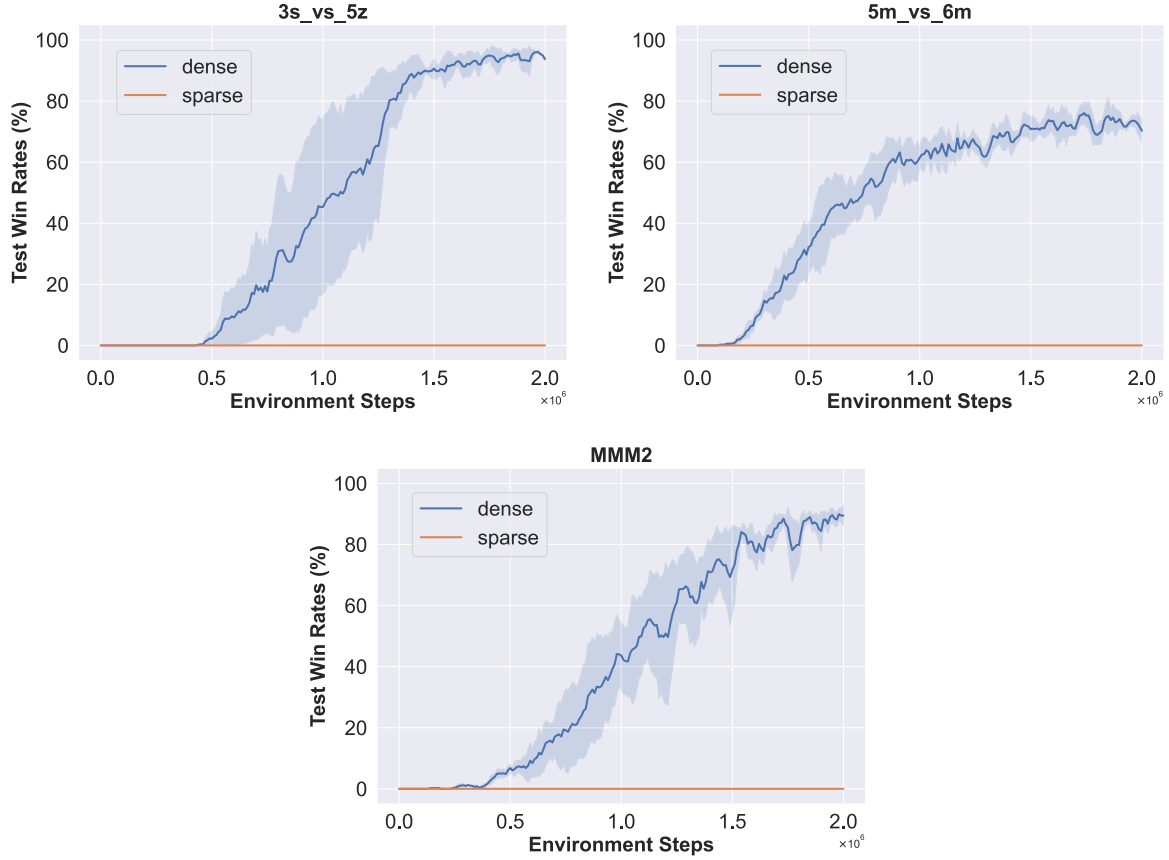


Fig. 16. The performance of MACE in scenarios with sparse reward.

with $\beta = 10$ rendering learning ineffective, confirming that excessive entropy regularization destabilizes policy optimization. For MMM2, both $\beta = 0.01$ and $\beta = 0.1$ yield high final performance (over 85% win rate), indicating greater robustness to exploration intensity. However, the effect of $\beta = 10$ is the best. This might be because in more complex scenarios, a larger policy entropy enables the agent to explore more useful state information, thereby converging to a better collaborative policy. Interestingly, in the 5m_vs_6m scenario, the influence of β on the collaborative policy is limited. In the homogeneous and nearly symmetrical scenarios, the additional benefits brought by the policy entropy are not significantly reflected, and the differences between different values of β do not exceed approximately 8%. To balance the advantage enhancement in various scenarios, we set $\beta = 0.1$ in the experiments of the paper.

5.4.8. The testing results on sparse reward setting

Although MACE has two different types of modules to enhance exploration, in reality, its success still depends on the accurate estimation of the centralized value function Q_{jt} . In other words, if the function estimated by Q_{jt} is not precise enough, it cannot provide an accurate update direction for the collaborative policy, and the value decomposition discrepancy-based intrinsic reward still cannot provide the correct intrinsic reward either, and may even cause the model to collapse. As shown in Fig. 16, in the environment of sparse rewards, MACE is also completely unable to learn the cooperative policies that can lead to victory, and it completely fails in the three scenarios of 3s_vs_5z, 5m_vs_6m, and MMM2. The possible reason for this is that in scenarios with sparse rewards, the interaction of agents cannot obtain useful reward information, thus their collaborative models cannot be effectively updated, resulting in Q_{jt} not having the correct valuation and thus being unable to conduct effective exploration. In this case, MACE will also be difficult to succeed.

5.4.9. Discussion: architectural capacity parity of MACE

To ensure fairness and transparency regarding architectural capacity, we standardize the backbone network configurations (e.g., DRQN recurrent layers and embedding dimensions) across all baseline methods (QMIX, WQMIX, QPLEX, ResQ, QTRAN, etc.) within the PyMARL framework. While MACE incorporates dual value estimators (Q_{jt} and Q_{tot}) and an action preference network, this architectural design is functionally driven by the need to compute value decomposition discrepancies for exploration guidance, rather than simply increasing model capacity. The core performance improvement stems from more accurate centralized value function estimation and the intrinsic reward derived from the discrepancy between unrestricted (Q_{jt}) and restricted (Q_{tot}) estimates, which effectively corrects value estimation bias. This is substantiated by our ablation studies in Section 5.4.2 and Section 5.4.3, where disabling the discrepancy-based intrinsic reward and preference mechanism led to significant performance declines (e.g., 12%-25% win rate drop in heterogeneous scenarios), even when the base network structure remained. Thus, compared to baselines like QTRAN's transformation constraints or QPLEX's duplex structure, the superiority of MACE is attributed to the effectiveness of the contrastive exploration strategy in guiding policy updates, rather than mere architectural expansion.

Indeed, similar to ResQ and WQMIX, MACE employs two separate network architectures to estimate Q_{jt} and Q_{tot} respectively. With the assistance of the action preference network, it can increase the policy entropy and thereby enhance the exploration of the state space by the collaborative model. The attention module included in the model architecture is mainly designed to better explore the correlation between the input features and the collaborative strategies, thereby enabling a more accurate estimation of the centralized value function. The conclusion that can be drawn is that more accurate value function estimation

and more thorough exploration of the state space are the fundamental reasons for the improvement in the efficiency of collaborative model strategy training. This is also the main purpose of the expansion and supplementation of the MACE architecture.

6. Conclusion

This paper proposes a novel method called MACE that combines collaborative policy with restricted representation capacity and higher representation capacity, aiming to balance the relationship between representation ability and the training efficiency of the value decomposition algorithms. MACE extends two centralized value function estimators, each with distinct advantages, using two different value decomposition methods and calculates the discrepancy between them to design an optimism objective in the function updates, aiming to reduce the estimation error of the restricted centralized network's value function. This extension enables MACE to attain both high training efficiency and complete centralized value function representation capacity. On the other hand, MACE also designs an action preference network that reshapes the agent's action selection mechanism with the goal of maximizing the average policy entropy, enabling the agent to choose more appropriate actions during collaboration, thus accelerating cooperation in the training process. The results obtained from the Matrix Games and SMAC, indicate that MACE not only gradually learns the correct optimal joint actions but also outperforms most baseline algorithms without the need for specific environment feature fusion modules, demonstrating the effectiveness of MACE in extending different types of value decomposition policies.

CRedit authorship contribution statement

Siying Wang: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Conceptualization; **Hongfei Du:** Writing – review & editing, Software, Methodology; **Chiyu Cai:** Writing – review & editing, Visualization; **Yang Zhou:** Writing – review & editing, Visualization; **Wenyu Chen:** Writing – review & editing, Supervision; **Jinliang Shao:** Writing – review & editing, Supervision, Resources, Funding acquisition; **Yuhua Cheng:** Writing – review & editing, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was supported in part by the [National Natural Science Foundation of China \(62273077\)](#), the Sichuan Science and Technology Program (2026NSFSC1483, 2024NSFJQ0013, 2025ZDZX0006), the Intelligent Perception and Control Key Laboratory of Sichuan Province (2025IPCZ02), and the New Cornerstone Science Foundation through the XPLOER PRIZE.

Appendix A.

A.1. Proof of Theorem 1

Proof. Under the setting of the MACE framework, based on the principle that neural networks can approximate any function, the unrestricted $Q_{jt}(\tau, \mathbf{u})$ can still represent any function. Here we need to explore the relationship between $Q_{jt}(\tau, \mathbf{u})$ and $Q_{tot}(\tau, \mathbf{u}) = V_{tot}(\tau, \mathbf{u}) + A_{tot}(\tau, \mathbf{u})$. We will show that $\arg \max_{\mathbf{u}} Q_{jt}(\tau, \mathbf{u}) = \bar{\mathbf{u}}$, where $\bar{u}^i = \arg \max_{u^i} Q_i(\tau^i, u^i)$ and $\bar{\mathbf{u}} = [\bar{u}^i]_{i=1}^n$. Thus

$$Q_{jt}(\tau, \bar{\mathbf{u}}) = Q_{tot}(\tau, \bar{\mathbf{u}})$$

$$\begin{aligned} &= V_{tot}(\tau) + A_{tot}(\tau, \bar{\mathbf{u}}) \\ &= \sum_{i=1}^n Q_i(\tau^i, \bar{u}^i) + \sum_{i=1}^n \rho_i(\tau^i, \bar{u}^i) A_i(\tau^i, \bar{u}^i) \\ &= \sum_{i=1}^n Q_i(\tau^i, \bar{u}^i) + \sum_{i=1}^n \rho_i(\tau^i, \bar{u}^i) (Q^i(\tau^i, \bar{u}^i) - V^i(\tau^i)) \\ &= \sum_{i=1}^n Q_i(\tau^i, \bar{u}^i) + \sum_{i=1}^n \rho_i(\tau^i, \bar{u}^i) (Q^i(\tau^i, \bar{u}^i) - Q^i(\tau^i, u^i)) \\ &= \sum_{i=1}^n Q_i(\tau^i, \bar{u}^i) \\ &\geq \sum_{i=1}^n Q_i(\tau^i, \bar{u}^i) - \sum_{i=1}^n \rho_i(\tau^i, u^i) (Q^i(\tau^i, \bar{u}^i) - Q^i(\tau^i, u^i)) \\ &= \sum_{i=1}^n Q_i(\tau^i, \bar{u}^i) - \sum_{i=1}^n \rho_i(\tau^i, u^i) (V^i(\tau^i) - Q^i(\tau^i, u^i)) \\ &= \sum_{i=1}^n Q_i(\tau^i, \bar{u}^i) + \sum_{i=1}^n \rho_i(\tau^i, u^i) (Q^i(\tau^i, u^i) - V^i(\tau^i)) \\ &= V_{tot}(\tau) + A_{tot}(\tau, \mathbf{u}) = Q_{jt}(\tau, \mathbf{u}) \end{aligned} \quad (\text{A.1})$$

The inequality above implies that $\bar{\mathbf{u}} = [\bar{u}^i]_{i=1}^n$ maximizes $Q_{jt}(\tau, \mathbf{u})$. Thus $[Q_i(\tau^i, u^i)]_{i=1}^n$ satisfies IGM for $Q_{jt}(\tau, \mathbf{u})$. This ends the proof. $\square$

A.2. Proof of Theorem 2

Before proving that the introduction of the value decomposition discrepancy of MACE can improve the policy convergence, we need to make some reasonable assumptions.

Settings and Assumptions:

- The interactive environment is a Markov decision process of finite state-joint action, which means (1) the state space $\mathcal{S}$ is finite, (2) the action space U^i of each agent i is limited, and the joint action space $\mathbf{U} = \prod_{i=1}^n U^i$ is also limited, (3) The transition function $\mathcal{P}(s'|s, \mathbf{u})$ and the reward function $r(s, \mathbf{u})$ are known and bounded.
- The global state is fully observable: the global state can be accessed during training with $s \in \mathcal{S}$.
- There exists a unique optimal joint policy for Q_{tot} that satisfies the fixed point theorem, which means there exists a unique optimal joint action function $\pi^*(s) = \mathbf{u}^*(s)$ such that it holds for all s :

$$Q^*(s, \mathbf{u}^*(s)) > Q^*(s, \mathbf{u}(s)), \quad \forall \mathbf{u}(s) \neq \mathbf{u}^*(s). \quad (\text{A.2})$$

Proof. For Q_{tot} 's update operator $\mathcal{T} : \mathbb{R}^{|\mathcal{S}||\mathcal{U}|} \rightarrow \mathbb{R}^{|\mathcal{S}||\mathcal{U}|}$ as:

$$\mathcal{T} Q_{tot}(s, \mathbf{u}) := r(s, \mathbf{u}) + \omega(Q_{jt}(s, \mathbf{u}) - Q_{tot}(s, \mathbf{u})) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot|s, \mathbf{u})} \left[\max_{\mathbf{u}'} Q_{tot}(s', \mathbf{u}') \right]. \quad (\text{A.3})$$

We rewrite the updating equation of operator $\mathcal{T}$, then we can obtain

$$\mathcal{T} Q_{tot}(s, \mathbf{u}) + \omega Q_{tot}(s, \mathbf{u}) = r(s, \mathbf{u}) + \omega Q_{jt}(s, \mathbf{u}) + \gamma \mathbb{E}_{s'} \left[\max_{\mathbf{u}'} Q_{tot}(s', \mathbf{u}') \right]. \quad (\text{A.4})$$

With the assumption that $Q_{tot}(s, \mathbf{u})$ satisfies the fixed point theorem (i.e. $\mathcal{T} Q_{tot}(s, \mathbf{u}) = Q_{tot}(s, \mathbf{u})$), then

$$Q_{tot}(s, \mathbf{u}) + \omega Q_{tot}(s, \mathbf{u}) = r(s, \mathbf{u}) + \omega Q_{jt}(s, \mathbf{u}) + \gamma \mathbb{E}_{s'} \left[\max_{\mathbf{u}'} Q_{tot}(s', \mathbf{u}') \right] \quad (\text{A.5})$$

which can be rewritten as

$$\begin{aligned} (1 + \omega) Q_{tot}(s, \mathbf{u}) &= r(s, \mathbf{u}) + \omega Q_{jt}(s, \mathbf{u}) + \gamma \mathbb{E}_{s'} \left[\max_{\mathbf{u}'} Q_{tot}(s', \mathbf{u}') \right] \\ \Rightarrow Q_{tot}(s, \mathbf{u}) &= \underbrace{\frac{r(s, \mathbf{u}) + \omega Q_{jt}(s, \mathbf{u})}{1 + \omega}}_{\tilde{r}(s, \mathbf{u})} + \underbrace{\frac{\gamma}{1 + \omega}}_{\tilde{\gamma}} \mathbb{E}_{s'} \left[\max_{\mathbf{u}'} Q_{tot}(s', \mathbf{u}') \right] \end{aligned} \quad (\text{A.6})$$

The above equation is a standard Bellman optimal equation, corresponding to an equivalent MDP with the reward function $\tilde{r}(s, \mathbf{u})$ and the discount factor $\tilde{\gamma}$. Since $\tilde{\gamma} = \frac{\gamma}{1+\omega} < 1$ with $\omega > 0$ and $\gamma < 1$, we have $\tilde{\gamma} < \gamma < 1$.

Next, we redefine a standard Bellman optimal operator $\mathcal{T}_{\tilde{\gamma}}$ as

$$\mathcal{T}_{\tilde{\gamma}} Q_{tot}(s, \mathbf{u}) = \tilde{r}(s, \mathbf{u}) + \tilde{\gamma} \mathbb{E}_{s'} \left[\max_{\mathbf{u}'} Q_{tot}(s', \mathbf{u}') \right] \quad (\text{A.7})$$

According to the fixed-point iteration theorem, in the case where $\tilde{r}(s, \mathbf{u})$ is bounded and $\tilde{\gamma} < 1$, $\mathcal{T}_{\tilde{\gamma}}$ is $\tilde{\gamma}$ -contraction under $\|\cdot\|_{\infty}$. Comparing Eqs. (A.6) and (A.7), we can conclude that the fixed point of $\mathcal{T}$ is equivalent to that of $\mathcal{T}_{\tilde{\gamma}}$. Then for ω :

- $\mathcal{T}$ has a unique fixed point $Q_{tot|w}^*(s, \mathbf{u})$.
- The Bellman iteration of the function $Q_{tot}(s, \mathbf{u})$ converges to $Q_{tot|w}^*(s, \mathbf{u}) = Q^*(s, \mathbf{u})$ with $Q_{tot}^{t+1}(s, \mathbf{u}) = \mathcal{T} Q_{tot}^t(s, \mathbf{u})$.
- The discounted factor is $\tilde{\gamma} = \frac{\gamma}{1+\omega} < 1$.

Then, we prove that when $Q_{jt}(s, \mathbf{u}) = Q^*(s, \mathbf{u})$, the fixed point $Q_{tot|w}^*(s, \mathbf{u})$ is the true optimal Q function. When $Q_{jt}(s, \mathbf{u}) = Q^*(s, \mathbf{u})$, we have

$$\tilde{r}(s, \mathbf{u}) = \frac{r(s, \mathbf{u}) + \omega Q^*(s, \mathbf{u})}{1 + \omega} \quad (\text{A.8})$$

with $Q^*(s, \mathbf{u}) = r(s, \mathbf{u}) + \gamma \mathbb{E}_{s'} [\max_{\mathbf{u}'} Q^*(s', \mathbf{u}')] we have$

$$\begin{aligned} \mathcal{T} Q_{tot}^*(s, \mathbf{u}) &= r(s, \mathbf{u}) + \omega (Q^*(s, \mathbf{u}) - Q_{tot}^*(s, \mathbf{u})) + \gamma \mathbb{E}_{s'} \left[\max_{\mathbf{u}'} Q_{tot}^*(s', \mathbf{u}') \right] \\ &= r(s, \mathbf{u}) + \gamma \mathbb{E}_{s'} \left[\max_{\mathbf{u}'} Q_{tot}^*(s', \mathbf{u}') \right] \\ &= Q_{tot}^*(s, \mathbf{u}) \end{aligned} \quad (\text{A.9})$$

Therefore, $Q_{tot}^*(s, \mathbf{u})$ is the fixed point of $\mathcal{T}$, due to the uniqueness of the fixed point, we have $Q_{tot|w}^*(s, \mathbf{u}) = Q_{tot}^*(s, \mathbf{u}) = Q^*(s, \mathbf{u})$.

Finally, for the approximation error bound of $Q_{tot|w}^*(s, \mathbf{u})$, let $\Delta = Q_{tot|w}^*(s, \mathbf{u}) - Q^*(s, \mathbf{u})$, $\delta = Q_{jt}(s, \mathbf{u}) - Q^*(s, \mathbf{u})$, from the fixed point equation we can get

$$\begin{aligned} \Delta &= \frac{r(s, \mathbf{u}) + \omega(Q^*(s, \mathbf{u}) + \delta)}{1 + \omega} + \frac{\gamma}{1 + \omega} \max_{\mathbf{u}'} Q_{tot|w}^*(s, \mathbf{u}) - (r(s, \mathbf{u}) + \gamma \max_{\mathbf{u}'} Q^*(s, \mathbf{u})) \\ &= \frac{\omega\delta}{1 + \omega} + \frac{\gamma}{1 + \omega} \max_{\mathbf{u}'} Q_{tot|w}^*(s, \mathbf{u}) - \gamma \max_{\mathbf{u}'} Q^*(s, \mathbf{u}) \end{aligned} \quad (\text{A.10})$$

and we have

$$(1 + \omega)\Delta = \omega\delta + \gamma \left(\max_{\mathbf{u}'} Q_{tot|w}^*(s, \mathbf{u}) - \max_{\mathbf{u}'} Q^*(s, \mathbf{u}) \right). \quad (\text{A.11})$$

By applying infinite norm $\|\cdot\|_{\infty}$ to both sides of Eq. A.11 and using the contraction mapping inequality $\|\mathcal{T} Q_{tot|w}^*(s, \mathbf{u}) - \mathcal{T} Q^*(s, \mathbf{u})\|_{\infty} \leq \|Q_{tot|w}^*(s, \mathbf{u}) - Q^*(s, \mathbf{u})\|_{\infty} = \|\Delta\|_{\infty}$, we have

$$\begin{aligned} (1 + \omega)\|\Delta\|_{\infty} &\leq \omega\|\delta\|_{\infty} + \gamma\|\Delta\|_{\infty} \\ \Rightarrow (1 + \omega - \gamma)\|\Delta\|_{\infty} &\leq \omega\|\delta\|_{\infty} \\ \Rightarrow \|Q_{tot|w}^*(s, \mathbf{u}) - Q^*(s, \mathbf{u})\|_{\infty} &\leq \frac{\omega}{1 + \omega - \gamma} \|Q_{jt}(s, \mathbf{u}) - Q^*(s, \mathbf{u})\|_{\infty}. \end{aligned} \quad (\text{A.12})$$

Due to $1 + \omega - \gamma > 1 - \gamma > 0 \Rightarrow 0 < \frac{\omega}{1 + \omega - \gamma} < 1$, this boundary is limited and reasonable, **which ends the proof.**

□

References

Angelotti, G., & Díaz-Rodríguez, N. (2023). Towards a more efficient computation of individual attribute and policy contribution for post-hoc explanation of cooperative multi-agent systems using myerson values. *Knowledge-Based Systems*, 260, 110189.

Böhmer, W., Kurin, V., & Whiteson, S. (2020). Deep coordination graphs. In *International conference on machine learning* (pp. 980–991). PMLR.

Chen, J., Li, T., Zhang, Y., You, T., Lu, Y., Tiwari, P., & Kumar, N. (2023). Global-and-local attention-based reinforcement learning for cooperative behaviour control of multiple UAVs. *IEEE Transactions on Vehicular Technology*, (pp. 1–13). <https://doi.org/10.1109/TVT.2023.3327571>

Ding, S., Du, W., Ding, L., Guo, L., Zhang, J., & An, B. (2023a). Multi-agent dueling q-learning with mean field and value decomposition. *Pattern Recognition*, 139, 109436.

Ding, S., Du, W., Ding, L., Zhang, J., Guo, L., & An, B. (2023b). Multiagent reinforcement learning with graphical mutual information maximization. *IEEE Transactions on Neural Networks and Learning Systems*.

Elfwing, S., Uchibe, E., & Doya, K. (2018). Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107, 3–11.

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018a). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning* (pp. 1861–1870). PMLR.

Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P. et al. (2018b). Soft actor-critic algorithms and applications. arXiv:1812.05905.

Hernandez-Leal, P., Kartal, B., & Taylor, M. E. (2019). A survey and critique of multi-agent deep reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 33(6), 750–797.

Hu, J., Wang, S., Jiang, S., & Wang, W. (2023). Rethinking the implementation tricks and monotonicity constraint in cooperative multi-agent reinforcement learning. In *The second blogpost track at ICLR 2023*.

Huang, A., Wang, Y., Sang, J., Wang, X., & Wang, Y. (2024). Dvf: multi-agent q-learning with difference value factorization. *Knowledge-Based Systems*, 286, 111422. <https://doi.org/10.1016/j.knsys.2024.111422>

Hüttenrauch, M., Šošić, A., & Neumann, G. (2017). Guided deep reinforcement learning for swarm systems. arXiv:1709.06011.

Jaques, N., Lazaridou, A., Hughes, E., Gulcehre, C., Ortega, P., Strouse, D. J., Leibo, J. Z., & De Freitas, N. (2019). Social influence as intrinsic motivation for multi-agent deep reinforcement learning. In *International conference on machine learning* (pp. 3040–3049). PMLR.

Jiang, J., Dun, C., Huang, T., & Lu, Z. (2018). Graph convolutional reinforcement learning. arXiv:1810.09202.

Khan, M. J., Ahmed, S. H., & Sukthankar, G. (2022). Transformer-based value function decomposition for cooperative multi-agent reinforcement learning in starcraft. In *Proceedings of the AAAI conference on artificial intelligence and interactive digital entertainment* (pp. 113–119). (vol. 18).

Lee, S., Seo, Y., Lee, K., Abbeel, P., & Shin, J. (2022). Offline-to-online reinforcement learning via balanced replay and pessimistic q-ensemble. In *Conference on robot learning* (pp. 1702–1712). PMLR.

Li, J., Kuang, K., Wang, B., Liu, F., Chen, L., Fan, C., Wu, F., & Xiao, J. (2022). Deconvolved value decomposition for multi-agent reinforcement learning. In *International conference on machine learning* (pp. 12843–12856). PMLR.

Li, P., Chen, S., Yuan, W., Hu, Z., & Chen, J. (2024). Vaos: Enhancing the stability of cooperative multi-agent policy learning. *Knowledge-Based Systems*, 304, 112474. <https://doi.org/10.1016/j.knsys.2024.112474>

Liang, S., & Srikant, R. (2017). Why deep neural networks for function approximation? In *International conference on learning representations*. <https://openreview.net/forum?id=SkpSIKlel>.

Liu, S., Liu, W., Chen, W., Tian, G., Chen, J., Tong, Y., Cao, J., & Liu, Y. (2023a). Learning multi-agent cooperation via considering actions of teammates. *IEEE Transactions on Neural Networks and Learning Systems*.

Liu, S., Zhou, Y., Song, J., Zheng, T., Chen, K., Zhu, T., Feng, Z., & Song, M. (2023b). Contrastive identity-aware learning for multi-agent value decomposition. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 11595–11603). (vol. 37).

Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. In *Proceedings of the 31st international conference on neural information processing systems, NeurIPS* (pp. 6382–6393).

Luo, Y., Wang, Y., Dong, K., Zhang, Q., Cheng, E., Sun, Z., & Song, B. (2023). Relay hindsight experience replay: Self-guided continual reinforcement learning for sequential object manipulation tasks with sparse rewards. *Neurocomputing*, 557, 126620.

Mei, Y., Zhou, H., Lan, T., Venkataramani, G., & Wei, P. (2023). Mac-po: Multi-agent experience replay via collective priority optimization. In *Proceedings of the 2023 international conference on autonomous agents and multiagent systems* (pp. 466–475).

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., & Ostrovski, G. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.

Naderialzadeh, N., Hung, F. H., Soleyman, S., & Khosla, D. (2020). Graph convolutional value decomposition in multi-agent reinforcement learning. arXiv:2010.04740.

Nguyen, T. T., Nguyen, N. D., & Nahavandi, S. (2020). Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications. *IEEE Transactions on Cybernetics*, 50(9), 3826–3839. <https://doi.org/10.1109/TCYB.2020.2977374>

Oliehoek, F. A., & Amato, C. (2016). A Concise Introduction to Decentralized POMDPs. Springer Briefs in Intelligent Systems. Springer.

Perrusquia, A., Yu, W., & Li, X. (2021). Multi-agent reinforcement learning for redundant robot control in task-space. *International Journal of Machine Learning and Cybernetics*, 12, 231–241.

Pu, Y., Wang, S., Yang, R., Yao, X., & Li, B. (2021). Decomposed soft actor-critic method for cooperative multi-agent reinforcement learning. arXiv:2104.06655.

Rashid, T., Faruqhar, G., Peng, B., & Whiteson, S. (2020a). Weighted qmix: Expanding monotonic value function factorisation for deep multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 33, 10199–10210.

Rashid, T., Samvelyan, M., De Witt, C. S., Faruqhar, G., Foerster, J., & Whiteson, S. (2020b). Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(178), 1–51.

- Rashid, T., Samvelyan, M., De Witt, C. S., Farquhar, G., Foerster, J., & Whiteson, S. (2020c). Monotonic value function factorisation for deep multi-agent reinforcement learning. *The Journal of Machine Learning Research*, 21(1), 7234–7284.
- Rashid, T., Samvelyan, M., Witt, C. S. d., Farquhar, G., Foerster, J. N., & Whiteson, S. (2018). QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th international conference on machine learning* (pp. 4292–4301). PMLR.
- Samvelyan, M., Rashid, T., Witt, C. S. d., Farquhar, G., Nardelli, N., Rudner, T. G. J., Hung, C.-M., Torr, P. H. S., Foerster, J. N., & Whiteson, S. (2019). The starcraft multi-agent challenge. In *Proceedings of the 18th international conference on autonomous agents and multiagent systems, AAMAS* (pp. 2186–2188).
- Shen, S., Qiu, M., Liu, J., Liu, W., Fu, Y., Liu, X., & Wang, C. (2022). Resq: A residual q function-based approach for multi-agent reinforcement learning value factorization. *Advances in Neural Information Processing Systems*, 35, 5471–5483.
- Son, K., Ahn, S., Reyes, R. D., Shin, J., & Yi, Y. (2020). Qtran + + : improved value transformation for cooperative multi-agent reinforcement learning. *arXiv:2006.12010*.
- Son, K., Kim, D., Kang, W. J., Hostallero, D. E., & Yi, Y. (2019). Qtran: Learning to factorize with transformation for cooperative multi-agent reinforcement learning. In *International conference on machine learning* (pp. 5887–5896). PMLR.
- Sunehag, P., Lever, G., Gruslys, A., Czarniecki, W. M., Zambaldi, V., Jaderberg, M., Lanctot, M., Sonnerat, N., Leibo, J. Z., Tuyls, K. et al. (2017). Value-decomposition networks for cooperative multi-agent learning. *arXiv:1706.05296*.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT press.
- Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (1999). Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12.
- Tan, M. (1993). Multi-agent reinforcement learning: Independent vs. cooperative agents. In *Proceedings of the 10th international conference on machine learning* (pp. 330–337). PMLR.
- Wan, (2023). Regularization-adapted anderson acceleration for multi-agent reinforcement learning. *Knowledge-Based Systems*, 275, 110709. <https://doi.org/10.1016/j.knsys.2023.110709>
- Wan, (2024). Enhancing collaboration in multi-agent reinforcement learning with correlated trajectories. *Knowledge-Based Systems*, 305, 112665. <https://doi.org/10.1016/j.knsys.2024.112665>
- Wang, J., Ren, Z., Liu, T., Yu, Y., & Zhang, C. (2020). Qplex: Duplex dueling multi-agent q-learning. *arXiv:2008.01062*.
- Wang, S., Li, X., Qu, H., & Chen, W. (2024). State augmentation via self-supervision in offline multiagent reinforcement learning. *IEEE Transactions on Cognitive and Developmental Systems*, 16(3), 1051–1062. <https://doi.org/10.1109/TCDS.2023.3326297>
- Wei, E., Wicke, D., Freelan, D., & Luke, S. (2018). Multiagent soft q-learning. *arXiv:1804.09817*.
- Wei, Q., Li, Y., Zhang, J., & Wang, F.-Y. (2022). Vgn: Value decomposition with graph attention networks for multiagent reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*.
- Wu, Q., Wu, J., Shen, J., Du, B., Telikani, A., Fahmideh, M., & Liang, C. (2022). Distributed agent-based deep reinforcement learning for large scale traffic signal control. *Knowledge-Based Systems*, 241, 108304.
- Xiao, B., Li, R., Wang, F., Peng, C., Wu, J., Zhao, Z., & Zhang, H. (2023a). Stochastic graph neural network-based value decomposition for multi-agent reinforcement learning in urban traffic control. In *2023 IEEE 97th vehicular technology conference (VTC2023-spring)* (pp. 1–7). IEEE.
- Xiao, J., Pisutsin, P., & Feroskhan, M. (2023b). Collaborative target search with a visual drone swarm: An adaptive curriculum embedded multistage reinforcement learning approach. *IEEE Transactions on Neural Networks and Learning Systems*, (pp. 1–15). <https://doi.org/10.1109/TNNLS.2023.3331370>
- Xie, T., Jiang, N., Wang, H., Xiong, C., & Bai, Y. (2021). Policy finetuning: Bridging sample-efficient offline and online reinforcement learning. *Advances in Neural Information Processing Systems*, 34, 27395–27407.
- Xu, X., Li, R., Zhao, Z., & Zhang, H. (2022). Stigmergic independent reinforcement learning for multiagent collaboration. *IEEE Transactions on Neural Networks and Learning Systems*, 33(9), 4285–4299. <https://doi.org/10.1109/TNNLS.2021.3056418>
- Xu, Z., Bai, Y., Zhang, B., Li, D., & Fan, G. (2023). Haven: Hierarchical cooperative multi-agent reinforcement learning with dual coordination mechanism. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 11735–11743). (vol. 37).
- Yang, S., Yang, B., Zeng, Z., & Kang, Z. (2023). Causal inference multi-agent reinforcement learning for traffic signal control. *Information Fusion*, 94, 243–256.
- Yang, Y., Hao, J., Liao, B., Shao, K., Chen, G., Liu, W., & Tang, H. (2020). Qatten: A general framework for cooperative multiagent reinforcement learning. *arXiv:2002.03939*.
- Zhang, H., Xu, W., & Yu, H. (2023a). Policy expansion for bridging offline-to-online reinforcement learning. *arXiv:2302.00935*.
- Zhang, J.-D., He, Z., Chan, W.-H., & Chow, C.-Y. (2023b). DeepMAG: Deep reinforcement learning with multi-agent graphs for flexible job shop scheduling. *Knowledge-Based Systems*, 259, 110083.
- Zhang, R., Zong, Q., Zhang, X., Dou, L., & Tian, B. (2023c). Game of drones: Multi-UAV pursuit-evasion game with online motion planning by deep reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*, 34(10), 7900–7909. <https://doi.org/10.1109/TNNLS.2022.3146976>
- Zhao, Z., Zhang, Y., Wang, S., Zhang, F., Zhang, M., & Chen, W. (2024a). Qdap: Downsizing adaptive policy for cooperative multi-agent reinforcement learning. *Knowledge-Based Systems*, 294, 111719. <https://doi.org/10.1016/j.knsys.2024.111719>
- Zhao, Z., Zhang, Y., Wang, S., Zhang, F., Zhang, M., & Chen, W. (2024b). Qdap: Downsizing adaptive policy for cooperative multi-agent reinforcement learning. *Knowledge-Based Systems*, 294, 111719.
- Zhou, W., Chen, D., Yan, J., Li, Z., Yin, H., & Ge, W. (2022). Multi-agent reinforcement learning for cooperative lane changing of connected and autonomous vehicles in mixed traffic. *Autonomous Intelligent Systems*, 2(1), 5.
- Zohar, R., Mannor, S., & Tennenholtz, G. (2022). Locality matters: A scalable value decomposition approach for cooperative multi-agent reinforcement learning. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 9278–9285). (vol. 36).